

Retrieval-Augmented Mixture of LoRA Experts for Uploadable Machine Learning

Ziyu Zhao, Leilei Gan, Guoyin Wang, Yuwei Hu, Tao Shen, Hongxia Yang, *Member, IEEE*,
Fei Wu, *Senior Member, IEEE*, Kun Kuang[†]

Abstract—Low-Rank Adaptation (LoRA) offers an effective solution for fine-tuning large language models (LLMs). Its modular nature allows the integration of diverse domain-specific LoRAs to enhance LLM capabilities. The emergence of open-source platforms like Huggingface has given rise to a new computational paradigm termed Uploadable Machine Learning (UML). In this framework, edge-side contributors train specialized LoRA adapters and then upload them to a central platform that leverages their plug-and-play nature to provide personalized services for downstream heterogeneous requests. Previous research on LoRA composition has centered on isolated tasks or static adapter selection during training, which does not adequately address the dynamic and expanding pool of LoRAs in UML nor sufficiently meet the needs of downstream heterogeneous requests. To bridge this gap, we propose Retrieval-Augmented Mixture of LoRA Experts (RAMoLE), a retrieve-then-compose framework that adaptively retrieves and composes multiple LoRAs according to the input prompts. RAMoLE contains three main components: firstly, identifying and retrieving LoRAs relevant to the given input through LoraRetriever; secondly, coordinating various dynamically retrieved LoRAs through a carefully designed on-the-fly MoLE mechanism; and thirdly, developing efficient batch inference to accommodate heterogeneous requests. The experimental results indicate that RAMoLE consistently outperforms the baselines, emphasizing its effectiveness and flexible scalability.

Index Terms—Data mining, text mining, natural language processing, and retrieval models.

I. INTRODUCTION

Recently, leveraging the principles of scaling laws [1], Large Language Models (LLMs) like openAI's GPT-3.5 [2], GPT-4 [3], Meta's LLaMA [4] and many other Foundation Models [5]–[8] have achieved notable success across various natural language processing (NLP) tasks [9], [10]. However, these models, primarily trained on diverse internet datasets, sometimes struggle in specialized areas. To address this, numerous studies [9], [11], [12] have explored fine-tuning LLMs using domain-specific data, thereby adapting them to fulfill specific needs. Due to the prohibitively high computation costs for fine-tuning LLMs on specific domains, there is a growing shift towards Parameter-Efficient Fine-Tuning (PEFT) [13]–[15], which only updates a small fraction of the

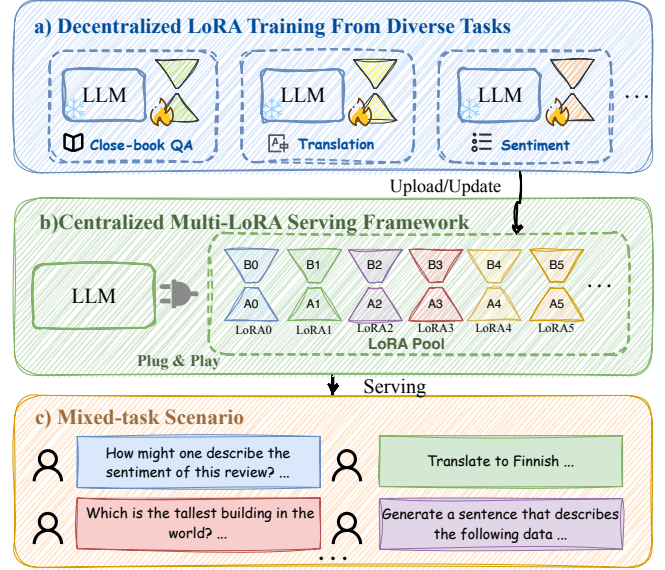


Figure 1. Illustration of UML. a) LoRAs from various domains and tasks aimed at enhancing specific capabilities of the LLM can be uploaded to or updated to the LoRA pool. b) The multi-LoRA serving framework aims to leverage the plug-and-play nature of LoRAs to offer comprehensive services. c) The downstream tasks, presented in a mixed-task form, require personalized expert routing.

model's parameters or integrates new trainable parameters that augment the model's capabilities.

As interest grows in open-source communities like Hugging Face¹, ModelScope², and Civit AI³, there is an increasing trend towards sharing numerous PEFT modules trained on a broad spectrum of domains and tasks across these platforms. Within these PEFT methods, LoRA has become the dominant fine-tuning method [16]. LoRA achieves efficiency by decomposing the weight matrices of a pre-trained model into low-rank factors, allowing it to adapt the model with minimal parameter changes. This approach not only significantly reduces computational load but also maintains or improves model performance through targeted updates. Its modularity and plug-and-play capabilities make it highly adaptable for various tasks and applications. This trend is giving rise to a new decentralized computing paradigm, which we refer to as *Uploadable Machine Learning* (UML) [16]–[18]. As shown in Fig.1, this new computing paradigm involves three

[†]Corresponding author.

Z. Zhao, L. Gan, Y. Hu, T. Shen F. Wu and K. Kuang are with the College of Computer Science and Technology, Zhejiang University, China. (E-mail: benzhuo.styx@gmail.com; leileigan@zju.edu.cn; hyw-luv@zju.edu.cn; tao.shen@zju.edu.cn; wufeif@zju.edu.cn; kunkuang@zju.edu.cn).

G. Wang and H. Yang are with ByteDance Inc., Seattle, USA. (E-mail: guoyinwang.duke@gmail.com; hx.yang@bytedance.com).

¹<https://huggingface.co/>

²<https://modelscope.cn/home>

³<https://civitai.com/>

components: a) The edge-side domain-specific LoRA contributors. These contributors train plug-and-play, domain-specific LoRA modules in a decentralized manner using local, domain-specific data. The trained LoRA parameters are then uploaded to a centralized service platform. b) Centralized Multi-LoRA Serving platform. The server side runs an LLM equipped with a dynamic pool of LoRAs. The purpose of the LoRA pool is to enhance the LLM’s capabilities across various domains, allowing it to deliver strong performance in a wider range of fields. The LLM leverages the plug-and-play capability of LoRA to dynamically load various LoRAs, allowing it to deliver personalized services tailored to diverse downstream requests. c) Mixed-task Downstream Requests. The downstream requests cover a diverse range of tasks and are in mixed-task form. The heterogeneous nature of these requests requires that the server side utilize the appropriate LoRA for different prompts to deliver personalized services.

The serving system for UML, designed to equip LLMs with a large pool of LoRAs for UML scenarios, faces two primary challenges. First, as the pool of LoRAs expands and evolves, developing efficient routing mechanisms for this dynamically changing collection becomes critical. This includes the need for zero-shot routing strategies specifically for newly uploaded LoRAs, ensuring they are integrated smoothly without prior adaptation. Second, the nature of downstream tasks in this paradigm typically involves a mixed-task format, which necessitates personalized routing of specific LoRAs for each input prompt. Unlike a one-size-fits-all approach, this requires a tailored solution that dynamically coordinates LoRA modules from various domains to effectively respond to the diverse and specific demands of user prompts. Platforms like ChatGPT and Gemini exemplify this scenario, highlighting the complexity of providing accurate and contextually appropriate responses in real time.

Recently, some work has focused on how to compose different LoRAs to enhance the model’s capabilities. Several research has explored the integration of the mixture of expert (MoE; [19], [20]) with LoRAs [11], [21]–[24]. However, these methods fix the selection of LoRAs during training, lacking the flexibility to dynamically update and scale in scenarios where the LoRA pool may consistently expand in the UML scenario. This rigidity violates the inherently dynamic and uploadable nature of decentralized training involving domain-specific LoRA experts. LoRAHub [25] and AdapterSoup [26] explore composing LoRAs for specific downstream tasks. However, these two methods provide a one-size-fits-all approach to downstream tasks, failing to accommodate the heterogeneous nature of diverse requests in a UML setting.

In this paper, we introduce a Retrieval-Augmented Mixture of Experts (RAMoLE), a retrieve-then-compose framework designed to exploit the plug-and-play nature of LoRA for UML. Our framework consists of three key components: (1) **Input-aware LoRA Retrieval:** The first step of our framework is aligning user inputs with the corresponding LoRAs through sentence embeddings and is further refined by an instruction fine-tuning [27], [28] for effective LoRA retrieval. Through the retriever, we achieve a more flexible LoRA routing mechanism, whose training stage is disentangled from

the training and inference of the LLM. (2) **On-the-fly Mixture of LoRA Experts:** After retrieving the corresponding LoRAs for each sample, we need to finely differentiate and compose these LoRAs for targeted generation. To this end, we propose a novel MoLE mechanism that involves training an additional RouterLoRA, which can be seamlessly integrated into the LLM in an off-the-shelf manner. This mechanism uses an attention mechanism to determine the weights corresponding to the retrieved LoRAs, thereby achieving a more flexible and dynamic MoLE architecture. It is worth noting that vanilla LoRA composition methods, which involve either averaging parameters or outputs, assign uniform weight to all loaded LoRAs. Such approaches may not allocate the most effective weights to the most suitable LoRA, potentially leading to suboptimal outcomes. In contrast, our proposed MoLE mechanism, utilizing a RouterLoRA, finely tunes the allocation of weights to each LoRA, dynamically differentiating each one to enhance overall performance (3) **Batch Inference of Multiple LoRAs:** Most previous work on the input-adaptive inference of LLMs does not support batch inference [26], [29]. To tackle the challenge of heterogeneous batched requests, we construct a unique LoRA mapping matrix for batch samples. This allows for tailored inferences through efficient matrix multiplication, ensuring each request activates its corresponding LoRAs while maintaining batch processing efficiency.

To assess the performance of RAMoLE, we established a UML evaluation benchmark comprising 48 LoRAs spanning a variety of natural language understanding and generation tasks. The experimental results underline the effectiveness of the proposed methods in serving both in-domain and out-of-domain downstream requests. Furthermore, the retrieval routing method demonstrates robust generalization capabilities: despite the retriever being trained on only 40% of the tasks, it effectively identifies the appropriate LoRAs for unseen tasks. Similarly, the RouterLoRA, also trained on just 40% of the tasks, successfully routes LoRAs for tasks it has not previously encountered, achieving commendable performance.

We note that a shorter conference version of this paper appeared in [17]. Our conference version of the paper did not explicitly define the entire scenario. This manuscript more thoroughly and comprehensively defines the setting as *Uploadable Machine Learning*. Additionally, the conference version simply applied the same weights for composing the retrieved LoRAs, an approach that may not allocate the most effective weights to the most appropriate LoRA, potentially resulting in suboptimal outcomes. In this manuscript, we introduce a novel on-the-fly MoLE method that employs a RouterLoRA to distinguish between each retrieved LoRA, enhancing performance in the UML scenario. Additionally, to accommodate the new MoLE mechanism, the batch inference mechanism has been adapted to facilitate a more efficient inference process.

Our contribution can be summarized as follows.

- We explore a new computing paradigm termed Uploadable Machine Learning, which entails managing a continuously expanding LoRA pool for serving heterogeneous downstream requests.

- We propose RAMoLE, a novel framework designed for the massive deployment of multiple LoRAs in response to personalized downstream requests. This framework consists of input-aware LoRA retrieval processes, strategic LoRA combinations, and a customized batch inference mechanism.
- We propose a novel MoE method that leverages a Router-LoRA as the gating function and assigns weights to each proposed LoRA through an attention mechanism. This approach enables RAMoLE to achieve more flexible and on-the-fly MoE, dynamically accommodating the retrieved LoRAs and generalizing to unseen LoRAs as a zero-shot router.
- Empirically, we show that our proposed RAMoLE framework outperforms other baselines in mixed-task scenarios. Moreover, LoraRetriever and the on-the-fly MoE mechanism achieve good generalization on unseen tasks and LoRAs as zero-shot routers.

II. RELATED WORK

A. Mixture of Experts

The Mixture of Experts (MoE) method combines various specialized sub-modules, guided by a gating network to tailor responses to different input types [19], [20], [30]–[32]. Some work [11], [21], [22], [32]–[35] focuses on using the MoE method for PEFT to achieve more effective and efficient model fine-tuning. Other work [23], [24] focuses on the use of MoE to post hoc coordinate existing LoRA experts without specifically training the experts' parameters. Specifically, AdaMix [21] introduces a mixture-of-adaptations approach for fine-tuning models efficiently by combining multiple adaptation methods to reduce parameter overhead. [34] demonstrates the efficacy of combining MoE with lightweight parameter-efficient tuning methods to enhance instruction tuning for large language models, achieving performance comparable to full fine-tuning while significantly reducing computational resource requirements. [22] enhances MoE by developing a highly parameter-efficient model that achieves performance on par with full fine-tuning while significantly reducing parameter usage. SiRA [33] introduces a sparse mixture approach to low-rank adaptation, improving the efficiency and performance of large language models by optimizing sparse computation and reducing overfitting. MoELoRA [11] leverages MoE and LoRA to efficiently fine-tune large language models for diverse medical tasks, addressing the issues of task variety and high computational cost. LoRAMoE [32] integrates MoE and LoRA to enhance large language models' capability in maintaining and applying world knowledge efficiently, significantly improving model performance while reducing computational requirements. Mixture-of-LoRAs [35] presents a method that leverages a mixture of LoRA modules to enhance the multitask performance of large language models, achieving significant efficiency in fine-tuning across various tasks. These works focus on integrating MoE and LoRA for more efficient fine-tuning of LLMs and reducing computational resources during inference. Our work is orthogonal to these approaches, as we investigate post-hoc LoRA combination, aligning with recent

trends in adapting a large LoRA pool for various downstream tasks.

MoLE [24] proposes an inference-time MoE method to enhance the parameter efficiency and flexibility of large language models, allowing for improved fine-tuning and adaptation across diverse tasks without significant computational overhead. SMEAR [23] improves the performance of mixture-of-experts (MoE) models by using a soft merging approach for expert parameters combined with adaptive routing, avoiding the inefficiencies of discrete routing decisions and resulting in better expert specialization and overall model efficiency. PATGOOSE [36] explores a framework for zero-shot learning where a model learns to dynamically route inputs among specialized experts, thereby achieving generalization to unseen tasks by leveraging the expertise of various pre-trained modules, significantly enhancing the model's adaptability and performance in zero-shot scenarios. [37] presents a modular approach for LLMs by constructing and reusing a library of LoRA modules, enabling efficient fine-tuning and adaptation for multiple tasks, thereby facilitating the creation of versatile and resource-efficient LLMs that can be easily customized for different applications. These methods necessitate that the router has encountered all LoRAs during training and requires the selection of LoRAs to be fixed at that time. This limitation prevents these methods from managing the continuously expanding LoRA pool in UML and from routing for unseen LoRAs.

B. Adapter Merging

In addition to model ensembling through the MoE, there is an increasing focus on aggregating adapters from different domains through the method of Adapter Merging. [38] proposes to compose these parameter-efficient modules through linear arithmetic operations in the weight space to integrate different module capabilities. AdapterSoup [26] aggregates different adapters in the parameter space, allowing large language models to adapt to new domains without additional training. LoRAHub [25] employs random sampling of LoRA parameters from various domains and tasks, followed by black-box optimization to learn the weights of different LoRA parameters without involving model gradient backpropagation. LoRA-Flow [39] dynamically fuses multiple Low-Rank Adaptation (LoRA) modules to enhance the generative capabilities of LLMs, enabling more efficient and effective task-specific fine-tuning without substantial computational overhead. π -Tuning [40] considers not only language model experts but also visual model experts. It obtains weights for different LoRAs through task embeddings for downstream tasks, similarly utilizing weighted averaging in the parameter space for targeted model parameter adaptation. LoRA-Composer [41] proposes a framework that utilizes Low-Rank Adaptation to enable multi-concept customization in training-free diffusion models, addressing the challenge of integrating multiple concept LoRAs for complex and varied image generation tasks. [42] explores Multi-LoRA Composition for combining multiple LoRA modules to improve the quality and efficiency of image generation, allowing for the seamless integration of various image attributes and styles.

These methods offer a one-size-fits-all solution for downstream tasks, which is not suitable for mixed-task scenarios that require personalized services. Moreover, adapter merging methods may encounter issues with parameter interference [43], which can severely degrade the performance of the merged model.

C. Personalized LoRA serving

[44] propose S-LoRA to discuss serving thousands of concurrent LoRA. The framework targets scenarios in which multiple tasks must be handled simultaneously without compromising the efficiency of the base models. [45] propose FLoRA, which enables efficient batching of diverse request types in the LoRA of foundation models. These studies discuss how to deploy or train personalized LoRAs. However, these methods can only utilize a single user-specified LoRA during inference, failing to fully leverage the combination of LoRAs from different tasks. Moreover, the primary focus of these discussions is on computational strategies in GPUs and training strategies, which are orthogonal to the routing strategies with which we are concerned.

III. PRELIMINARIES

This section begins with a concise introduction to the Low-Rank Adaptation and Mixture of LoRA Experts, followed by a detailed formalization of our *Uploadable Machine Learning* setting.

A. Low-Rank Adaptation

Directly fine-tuning large language models with all parameters is computationally intensive and is not feasible in low-resource scenarios. Based on the idea that only a small number of low-rank parameters need to be fine-tuned for sufficient performance in new domains, [15] proposed the Low-Rank Adaptation, where the LoRA module can be combined with the pre-trained parameters in parallel for efficient inference.

Specifically, given pre-trained weights $W_0 \in \mathbb{R}^{d \times k}$ of a sub-module of LLM, the LoRA adds an extra trainable weight matrix as $W_0 + \Delta W = W_0 + BA$, where ΔW can be decomposed into two smaller matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, where r stands for the rank of ΔW and the rank $r \ll \min(d, k)$. The forward pass for a layer $x' = W_0 x$ can be modified as follows:

$$x' = W_0 x + \Delta W x = W_0 x + B A x, \quad (1)$$

where $x \in \mathbb{R}^d$ is the input and the $x' \in \mathbb{R}^d$ denote the output.

B. Mixture of LoRA Experts

The mixture of LoRA Experts (MoLE) is an efficient ensemble approach that enhances model scalability by selectively activating only a subset of parameters, thus increasing the model's capacity without incurring additional computational costs. Recently, several studies have explored the integration of Low-Rank Adaptation (LoRA) with MoE for diverse applications. Unlike traditional transformer models, each MoE

layer with the original parameter W consists of k independent LoRA experts $\{\Delta W_i\}_{i=1}^k$, along with a gating function $G(\cdot)$ that models the probability distribution to determine the weighting of the outputs of these experts. The output of a MoLE layer can be given by:

$$x' = W_0 x + \sum_{i=1}^k G(x)_i \cdot \Delta W_i x. \quad (2)$$

Here, $G(x)$ is a k -dimensional output of the gating network. A normal way of implementing the gating network is in the following way:

$$G(x)_i := \exp(x \cdot e_i) / \sum_{j=1}^k \exp(x \cdot e_j), \quad (3)$$

where e_i denotes the trainable embedding of the expert ΔW_i . However, such a routing mechanism is not suitable for the UML scenario, in which the LoRA pool often changes dynamically. This dynamic nature necessitates continual updates to the Expert Embedding based on modifications to LoRA. Furthermore, newly added LoRAs, which were not included during the initial training phase, are unable to participate in routing due to the lack of prior training.

C. Problem Formulation of Uploadable Machine Learning

In this part, we give a formal definition of Uploadable Machine Learning. The UML framework comprises three aspects as shown in Fig.1: upstream Edge-side LoRA contributors, the Multi-LoRA serving system on the service side, and downstream heterogeneous requests.

a) *Edge-side Contributors.*: Given an original LLM L , suppose we have k edge-side contributors. Each contributor trains a LoRA module based on its domain dataset, resulting in a set of k LoRAs, $\Phi = \{\phi_1, \phi_2, \dots, \phi_t, \phi'_{t+1}, \dots, \phi'_k\}$, where each LoRA ϕ_i is trained on its corresponding task T_i . The LoRAs ϕ'_i denote newly uploaded LoRAs that were not visible during the training phase.

b) *Multi-LoRA Serving System.*: With a LoRA pool Φ for the LLM L , the serving process can be written as:

$$y = F(g(\Phi, x), x, \theta, \gamma), \quad (4)$$

where θ denotes the original parameters of LLM, $g(\Phi, x)$ represents the input-aware LoRA retrieval process, and returns a set of retrieved LoRAs Φ_i . $F(\Phi_i, x_i, \theta)$ depicts the LoRA composition process that integrates the retrieved LoRAs as a plug-in to the original LLM. γ denotes the parameters of the gating function that differentiate the contribution of the retrieved LoRAs Φ_i . It is worth noting that during training, the parameters of the retriever $g(\cdot)$ and the gating function γ are trained on a set $\mathcal{T}_{train} = \{T_1, \dots, T_t\}$, leaving the tasks $\{T'_{t+1}, \dots, T'_k\}$ unseen for evaluate the generalization for the routing mechanism for unseen LoRAs.

c) *Downstream Heterogeneous Requests.*: The downstream requests are in mixed task form, where mixed task input can be formulated as $T_{mix} = \{x \mid x \in T_1 \vee T_2 \vee \dots \vee T_k\}$ where $\vee$ represents the logical disjunction operator. Given the heterogeneous nature of downstream requests, the serving

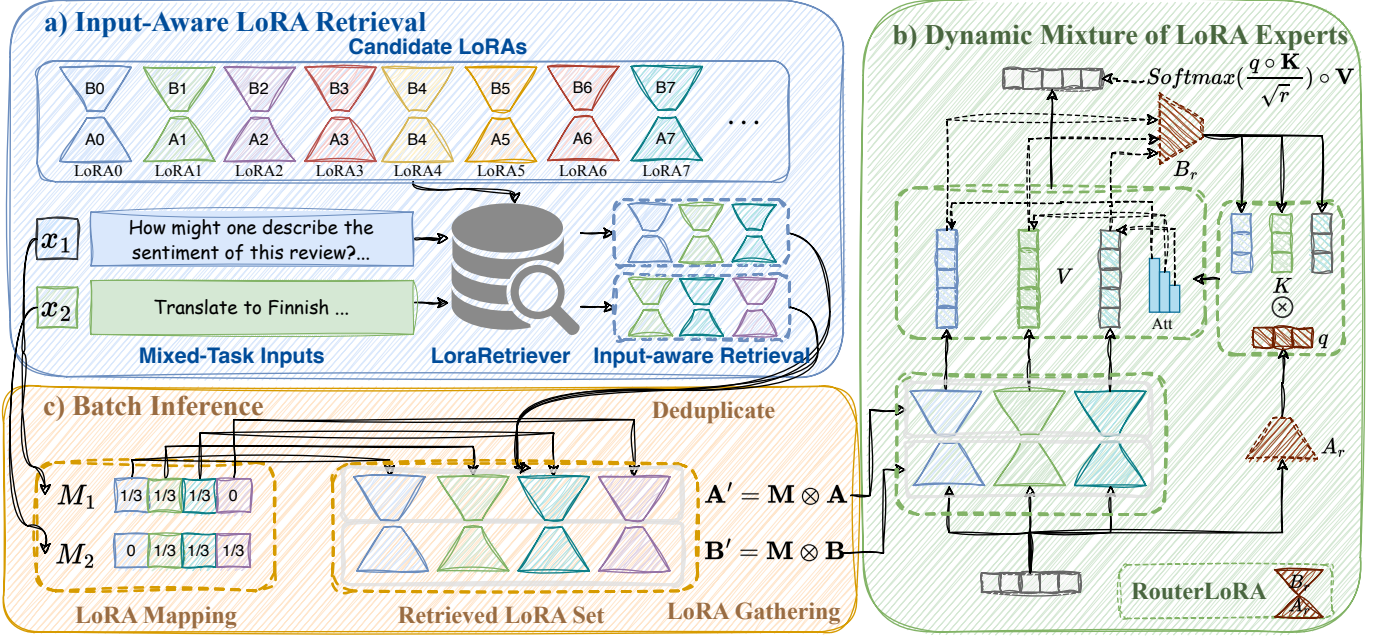


Figure 2. The RAMoLE Framework. This framework, equipped with a pool of candidate LoRAs from various domains/tasks, is designed to offer personalized services tailored to the input provided. It begins by executing an input-aware LoRA retrieval process aimed at identifying LoRAs corresponding to tasks analogous to the input (§IV-A). Subsequently, it employs a specialized LoRA composition mechanism to efficiently utilize the retrieved LoRAs (§IV-B). By constructing a LoRA mapping matrix for batch inputs, the framework facilitates effective batch inference (§IV-C).

process should provide personalized service for each input prompt.

IV. RETRIEVAL-AUGMENTED MIXTURE OF LoRA EXPERTS FRAMEWORK

In this section, we describe the RAMoLE framework as shown in Fig.2 for serving multi-LoRAs in UML. This framework contains three major components: the input-aware LoRA retrieval module (§IV-A), the On-the-fly MoLE module (§IV-B), and the batch inference strategy (§IV-C).

A. Input-Aware LoRA Retrieval

To manage a large pool of LoRAs, the initial stage of our framework involves constructing a LoraRetriever specifically designed to efficiently retrieve the appropriate LoRAs for each input, particularly in scenarios where LoRAs are dynamically updated. However, existing approaches fall short of accurately identifying LoRAs under such conditions. MoE-based methods [23], [24] struggle to generalize when new LoRAs are introduced due to the fixed selection of LoRAs established during router training. Retrieval methods such as sentence embedding [46], [47] or task embedding [48], [49] fail to map both samples and LoRA into a shared embedding space, limiting their effectiveness in input-aware LoRA retrieval.

To achieve this goal, we propose to train a retriever via instruction fine-tuning [27], [28], namely LoraRetriever, which can retrieve suitable LoRAs from a massive LoRA pool for a given input sample. The fundamental concept behind LoraRetriever comprises two main steps: (i) First, to embed different task-specific LoRAs into embedding space for facilitating retrieval, we posit that each LoRA can be

represented by some data points, which can be obtained by randomly choosing a dozen samples from the training dataset. Then we average their instruction embeddings to represent the embedding of each LoRA. (ii) To improve generalization for unseen LoRAs in LoRA retrieving, we train the retriever through instruction fine-tuning [5], [27] on a subset of all tasks. Training on a small subset of tasks is designed to simulate scenarios involving the integration of new LoRAs, thereby underscoring our method’s generalization abilities via instruction fine-tuning. These two strategies enable the effective use of limited data distributions for input-aware retrieval and can be generalized to unseen LoRAs.

Formally, with a sentence-embedding model E , input sequence x , and the instruction I for embedding purposes, the instructed embedding can be formulated as $E(I \oplus x)$, where $\oplus$ denotes the concatenation operation. In order to allow the embedding to capture the similarity between different tasks, the instruction is expressed as “Represent the sentence for similar task retrieval”. Each LoRA module is embedded with m randomly selected domain-specific samples, expressed as $E(\phi) = \frac{1}{m} \sum_{i=1}^m E(I \oplus x_{i\phi})$. This embedding method integrates both sample-wise and LoRA-module-wise embeddings, facilitating the calculation of similarity between an individual sample and a LoRA module. For measuring the similarity between LoRA module ϕ and the input sequence x , following [47], we leverage the cosine similarity between the LoraRetriever embeddings: $s(x, \phi, I) = \cos(E(I \oplus x), E(\phi))$.

To improve LoRA retrieval by the retriever and broaden its generalization to unseen LoRAs, we train the embedding model E through instruction fine-tuning on a small subset of tasks. To prevent the need to access new samples, we use previously employed samples for embedding LoRAs as our

training data. Consider t distinct training tasks, represented as $\mathcal{T}_{train} = \{T_1, \dots, T_t\}$. Following [47], the training dataset $\mathcal{D}$ comprises paired samples (x_i, x_i^+) , where each x_i is a sample from a task $T_i \in \mathcal{T}_{train}$, and a positive sample x_i^+ is randomly selected from the same task T_i . To complement each positive pair, we randomly select p negative pairs $(x_i, x_{ij}^-)_{j=1}^p$, ensuring that x_{ij}^- is sourced from tasks outside of T_i , thereby $x_{ij}^- \notin T_i$. The training process is achieved through a contrastive loss [47], [50], [51] defined as follows:

$$\mathcal{L} = \frac{e^{s(x_i, x_i^+, I)/\gamma}}{e^{s(x_i, x_i^+, I)/\gamma} + \sum_{j=1}^p e^{s(x_i, x_{ij}^-, I)/\gamma}},$$

where γ is the softmax temperature.

During the LoRA retrieval phase, the top- k LoRAs are retrieved according to their similarity to the input x . This process can be formulated as follows:

$$g(x_i, \Phi) := \Phi_i = \text{TopK}\{s(\phi_j, x_i, I), \phi_j \in \Phi\}.$$

B. On-the-fly Mixture of LoRA Experts

After retrieving the top- k LoRAs, Φ_i , for an input x_i , we proceed to integrate these LoRAs into the LLM with parameter θ . This section delves into the proposed on-the-fly Mixture of LoRA Experts (DMoLE) mechanism for LoRA composition. We start by introducing the vanilla LoRA composition strategies as described in [17], and illustrate the limitations of these vanilla methods. Subsequently, we offer a formal description of the proposed on-the-fly MoLE mechanism, which incorporates an additional LoRA serving as a router, namely RouterLoRA. Its primary purpose is to learn to route among retrieved LoRAs, effectively using cross-attention to assign weights to each one and decouple the routing process from specific LoRA modules.

1) *Vanilla LoRA Composition*: The vanilla LoRA composition comprises two training-free methods: the Mixture of LoRAs and the Fusion of LoRAs.

a) *Mixture of LoRAs*: The mixture of LoRAs strategy involves the aggregation of the outputs of each submodule within the assembled LoRAs. Let us denote $\mathbf{A} = \{A_1, A_2, \dots, A_n\}$ and $\mathbf{B} = \{B_1, B_2, \dots, B_n\}$ as the sets representing submodules within n LoRAs. For an input x_i , the output derived from the mixture of LoRAs can be expressed as:

$$x'_i = \frac{1}{n} \sum_{j=1}^n B_j A_j x_i, \quad (5)$$

where x'_i denotes the output. This process signifies the integration of each LoRA module's output, effectively blending their contributions to form a unified output.

b) *Fusion of LoRAs*: In contrast to the Mixture method, which combines the output of different LoRAs, fusing the parameters of these LoRAs presents an alternative composition strategy. Let the parameters of each LoRA ϕ_i be denoted by Θ_i . The parameter of the fused LoRA is then represented as $\Theta_{\text{fusion}} = \frac{1}{k} \sum_{j=1}^k \Theta_j$. This formulation allows the fused parameter to function akin to a single LoRA.

c) *Limitation of Vanilla Composition*: Since the LoRAs for different tasks are trained independently, the model is significantly affected by data heterogeneity during the parameter fusion process. This results in catastrophic collapses in performance on respective tasks after fusion due to parameter interference, a phenomenon that becomes more pronounced with the increasing number of composed LoRAs [17]. Consequently, the Fusion of LoRAs method is not suitable for application in scenarios involving UML. On the other hand, the mixture of LoRAs method, which averages the outputs of different LoRAs, achieves better performance, especially on unknown tasks. However, because it uniformly assigns the same weight to the output of each LoRA without differentiation, this method fails to fully leverage the most optimal LoRA for the task, leading to a suboptimal model performance.

2) *On-the-fly Mixture of LoRA Experts with RouterLoRA*: To allocate weights for the retrieved top- k LoRAs Φ_i in layer-level routing, the most straightforward idea is to train an additional router using the MoLE method to assign weights to each LoRA. However, since the traditional MoLE method requires fixing the selection of LoRAs during training and ensuring consistency between the LoRAs loaded during training and inference, it is unable to adapt to the dynamically changing LoRA pool in UML scenarios, making it unsuitable for such scenarios. To address this challenge, we propose a novel MoLE mechanism that includes an additional LoRA module specifically designed for *learning to route*. This module leverages an attention mechanism to dynamically assign weights to different LoRAs, optimizing the routing process. In this way, we decouple the router from the LoRA module for routing, achieving a more dynamic MoLE mechanism that can effectively generalize to newly uploaded LoRAs.

Formally, consider $\mathbf{A} = \{A_1, A_2, \dots, A_n\}$ and $\mathbf{B} = \{B_1, B_2, \dots, B_n\}$ as the sets of top- n LoRAs retrieved for a layer. A RouterLoRA, with parameters A_r and B_r , is integrated into the LLM. It functions as a router to determine the appropriate weights for each plugged-in LoRA module. The entire routing process operates through a mechanism akin to cross-attention operation. Initially, for an input x , it is processed by each LoRA module (A_i, B_i) to generate

$$v_i = B_i A_i x, \quad (6)$$

which can be seen as a transformation of x through a module-specific linear transformation followed by another module-specific transformation. Furthermore, the query vector q is derived from the router's parameters using:

$$q = A_r x, \quad (7)$$

where $q \in \mathbb{R}^{b \times l \times r}$ (assuming b is the batch size, l is the sequence length, and r is the dimensionality of the query vector). Each i -th LoRA module contributes a key vector k_i , calculated as:

$$k_i = B_r^T v_i, \quad (8)$$

where $k \in \mathbb{R}^{b \times l \times r}$. This transformation aims to map the output v_i into a space where it can be compared against the query q .

The next step involves calculating the attention weights. Each k_i is compared to q using a dot product, resulting in a

score that measures the alignment or relevance of each LoRA output to the query. These scores are then normalized through a softmax function to form attention weights:

$$s_i = \langle q, k_i \rangle / \sqrt{r} \quad (9)$$

$$\alpha = \text{Softmax}(s_1, s_2, \dots, s_n). \quad (10)$$

These attention weights α_i dictate the importance of each v_i in the final representation. The weighted sum of all v_i , modulated by α_i , produces the output of the RouterLoRA:

$$x' = \sum_{i=1}^k \alpha_i v_i. \quad (11)$$

This output x' then serves as the adjusted representation for the original input x within the context of the LLM, reflecting a dynamically weighted integration of different transformations offered by the LoRA modules. Thus, RouterLoRA employs an attention mechanism to adaptively route and differentiate LoRA modules across different layers, resulting in more fine-grained routing.

a) *Training of RouterLoRA*: Similar to the training process of LoraRetriever, here we leverage t distinct training tasks, represented as $\mathcal{T}_{train} = \{T_1, \dots, T_t\}$, and the training set $\mathcal{D}_{train} = \{(x, y) \in \mathcal{T}_{train}\}$. To test the generalization of the Dynamic MoLE mechanism, we train on only 40% of the LoRAs in the LoRA pool and perform zero-shot routing on the remaining LoRAs. And the training set of LoRAs can be denoted as $\Phi_{train} = \{\phi_1, \dots, \phi_t\}$. Besides, we denote the parameters of the RouterLoRA as γ . The training loss can be formulated in the following way:

$$\mathcal{L}_{train} = \frac{1}{|\mathcal{D}_{train}|} \sum_{x, y \in \mathcal{D}_{train}} \mathcal{L}(f(x; \Phi_{train}, \gamma, \theta), y). \quad (12)$$

In this formulation, only γ is trainable, focusing the training process on optimizing the router's parameters while keeping the parameters of individual LoRAs fixed. Through this targeted training, we successfully developed RouterLoRA to act as a router, capable of distinguishing between LoRA experts based on different hidden states, thus facilitating a more flexible and dynamic routing mechanism.

b) *Random Dropout LoRA modules for Improving Generalization.*: Loading all LoRAs directly into the model and training RouterLoRA on specific tasks significantly improves performance in in-distribution (IID) scenarios by efficiently identifying and routing to the optimal LoRA module. However, this method is less effective in out-of-distribution (OOD) scenarios, where no single LoRA perfectly fits the inference needs of downstream tasks. In these cases, combining different LoRAs is crucial for enhancing zero-shot generalization. Direct training of RouterLoRA often overlooks the complexities of unseen tasks, focusing mainly on optimizing performance within familiar IID tasks and neglecting the variability of OOD scenarios. Consequently, the model lacks the adaptability and robustness needed for effective generalization in new, unanticipated contexts, leading to suboptimal zero-shot performance. To mitigate this issue, we propose a strategy of randomly dropout LoRA modules during the training process to enhance our method's performance in OOD scenarios. Formally, we

redefine the training objective to incorporate module-level dropout as follows:

$$\mathcal{L}_{train} = \frac{1}{|\mathcal{D}_{train}|} \sum_{x, y \in \mathcal{D}_{train}} \mathcal{L}(f(x; \text{Dropout}(\Phi_{train}, p), \gamma, \theta), y), \quad (13)$$

where p denotes the dropout rate, here, the function $\text{Dropout}(\Phi_{train}, p)$ selectively deactivates a subset of modules Φ_{train} at a rate p , enabling us to train a more robust model by introducing variability and reducing overfitting.

C. Batch Inference of Multiple LoRAs

Implementing batch inference in the presence of multiple LoRAs and diverse composition diagrams poses a significant technical challenge. To address this, we introduce a unique approach for batch inference. Our method involves processing a batch of samples denoted as $X \in \mathbb{R}^{b \times l \times d}$, where b , l , and d denote the batch size, sequence length, and sample dimensionality, respectively. For each input x_i and its retrieved LoRAs Φ_i within the same batch, we aggregate these LoRAs into a collective set denoted by Φ_B . To ensure the uniqueness of Φ_B , we eliminate duplicates, mindful of the possibility that retrieved LoRAs may overlap across different samples. The resulting set Φ_B comprises p unique LoRAs, where $p \leq bk$. For every sample x_i , a p dimension mapping vector M_i is generated, which specifies the indices of its corresponding LoRAs within Φ_B .

The LoRA mapping vectors are combined into a matrix $M \in \mathbb{R}^{b \times p}$. The parameters of a submodule in LoRA can be denoted as A and B , and are concatenated within the batched LoRAs Φ_B to obtain $A \in \mathbb{R}^{p \times r \times d}$ and $B \in \mathbb{R}^{p \times d \times r}$. We first gather the LoRA modules for each input within the batch through $A' = M \otimes A$ and $B' = M \otimes B$ where $\otimes$ denotes the gather operation. By gathering the corresponding parameters for each input, we obtain the matrix $A' \in \mathbb{R}^{b \times k \times r \times d}$ and $B' \in \mathbb{R}^{b \times k \times d \times r}$. Then we can obtain the value matrix V through $V = B' \circ A' \circ X$, where $V \in \mathbb{R}^{k \times b \times l \times d}$ and we extend the symbol $\circ$ to denote potential broadcasting as [45]. Then the key matrix is computed through the RouterLoRA in the following way:

$$q = A_r \circ X \quad (14)$$

$$K = B_R^T \circ V \quad (15)$$

where $q \in \mathbb{R}^{1 \times b \times l \times r}$ and $K \in \mathbb{R}^{k \times b \times l \times r}$. The output is computed by $X' = \text{Softmax}(\frac{q \circ K}{\sqrt{r}}) \circ V$. In this way, we successfully perform personalized inference for each input sample, allowing efficient batch inference.

V. EXPERIMENTS

This section outlines the evaluation framework for assessing different approaches in our Uploadable Machine Learning setting. We conducted a detailed comparison of the performance differences of various LoRA composition strategies in different settings, to demonstrate the superiority of the proposed DAMoLE method. Furthermore, a comprehensive analysis of the proposed RAMoLE framework is presented.

Table I

THE AVERAGE PERFORMANCE OF EACH TASK CLUSTER. "IID" MEANS THAT RAMoLE CAN ACCESS ANY LoRA FOR EVERY TEST SAMPLE, ENCOMPASSING THE LoRA SPECIFIC TO THE SAMPLE'S TASK. "OOD" INDICATES THAT FOR EACH TEST SAMPLE, WE MASK THE LoRA ASSOCIATED WITH ITS SPECIFIC TASK DURING THE RETRIEVAL PHASE. CONSEQUENTLY, NO SAMPLE CAN ACCESS ITS IDEAL LoRA, ALLOWING US TO ASSESS THE RAMoLE'S CROSS-TASK GENERALIZATION CAPABILITY. THE PERFORMANCE OF PERFECTLY SELECTED CORRESPONDING LoRA FOR EACH SAMPLE IS COLORED IN GRAY. WE HAVE BOLDED THE BEST PERFORMANCE OF EACH TASK AND UNDERLINED THE BEST PERFORMANCE IN THE "OOD" SETTING.

Task	Perfect Selection	RAMoLE		Selection		Fusion		Mixture		MoE	MoE	MoE	SME-AR	Adapter	LoRA
		IID	OOD	IID	OOD	IID	OOD	IID	OOD	Top1	Top3	Soft	AR	Soup	Hub
w/ Llama2-7b															
Struct to Text _{Rouge-1}	59.1	58.4	<u>50.0</u>	56.8	45.2	44.5	41.0	51.2	45.3	41.3	41.8	43.2	43.3	3.5	31.9
Struct to Text _{Rouge-2}	36.1	32.6	<u>26.1</u>	33.6	23.2	22.6	20.2	26.3	22.9	19.3	19.9	20.7	21.3	0.9	15.1
Struct to Text _{Rouge-l}	48.6	51.9	<u>44.6</u>	46.4	35.3	34.5	31.7	41.0	<u>35.5</u>	32.6	32.8	33.8	33.9	3.3	24.9
Translation _{BLEU}	13.1	12.8	11.9	12.8	12.0	12.2	<u>12.3</u>	12.8	12.2	9.5	10.5	10.7	11.0	1.4	8.5
COMMONSENSE	62.5	58.5	55.0	55.5	46.0	51.0	48.0	61.5	50.0	54.5	52.0	51.5	50.0	46.0	17.5
SENTIMENT	90.0	90.5	92.0	89.5	89.0	79.0	78.5	89.5	90.5	70.0	75.0	74.5	74.0	73.5	0.5
READING Comp.	67.3	52.0	47.7	51.7	40.3	47.3	45.0	51.3	47.3	48.7	47.7	48.7	45.7	40.7	2.7
CLOSE-BOOK QA	45.0	46.0	45.5	40.0	43.0	41.0	37.5	45.0	<u>48.5</u>	40.5	38.5	40.0	32.0	31.5	1.0
COREFERENCE	52.0	61.0	<u>52.0</u>	50.0	46.0	47.0	<u>53.0</u>	63.0	49.0	61.0	59.0	57.0	58.0	43.0	1.0
READ. COOMP. W/ COM	69.0	64.0	<u>42.0</u>	69.0	30.0	35.0	19.0	46.0	40.0	31.0	29.0	29.0	23.0	14.0	3.0
PARAPHRASE	65.5	57.0	<u>46.5</u>	58.0	45.5	45.5	44.0	56.5	45.5	42.0	38.5	36.0	34.5	46.5	1.0
NLI	72.3	65.9	66.2	70.0	60.6	51.4	53.8	67.9	<u>64.3</u>	50.3	49.6	48.3	50.8	62.4	10.5
Overall	55.4	51.8	<u>47.7</u>	51.2	43.0	41.6	40.2	49.8	45.6	40.3	39.9	39.8	39.1	32.4	10.1
w/ Llama2-13b															
Struct to Text _{Rouge-1}	65.4	60.7	49.6	62.6	49.4	52.7	49.7	57.7	52.1	46.8	47.0	48.5	48.3	7.1	39.3
Struct to Text _{Rouge-2}	40.8	35.1	26.7	38.2	25.8	29.2	26.8	32.6	28.1	24.5	25.1	25.7	25.2	2.5	20.7
Struct to Text _{Rouge-l}	58.7	53.8	42.6	56.0	42.9	45.9	43.2	50.8	45.4	41.1	41.9	42.7	42.2	6.4	34.6
Translation _{BLEU}	12.9	14.0	14.0	12.9	12.7	14.6	14.1	14.6	14.1	11.8	12.4	11.9	12.4	0.8	10.2
COMMONSENSE	69.5	69.5	68.0	59.0	47.5	61.0	56.0	64.0	60.5	65.0	66.0	64.0	61.0	17.5	34.0
SENTIMENT	90.0	91.5	91.5	90.5	91.0	87.0	83.5	91.5	<u>91.5</u>	90.0	89.5	90.0	89.0	79.5	11.0
READING Comp.	76.0	62.3	<u>54.3</u>	60.3	48.0	56.7	49.3	60.3	51.3	53.7	53.3	52.3	51.3	48.7	3.3
CLOSE-BOOK QA	64.0	61.0	56.0	60.0	53.0	62.0	58.0	63.0	<u>61.0</u>	59.5	57.5	58.5	57.5	34.5	6.5
COREFERENCE	74.0	76.0	60.0	75.0	65.0	55.0	59.0	76.0	<u>64.0</u>	61.0	62.0	56.0	57.0	55.0	10.0
READ. COOMP. W/ COM	82.0	78.0	57.0	80.0	33.0	57.0	49.0	78.0	<u>58.0</u>	51.0	48.0	49.0	49.0	13.0	14.0
PARAPHRASE	77.5	67.0	<u>60.5</u>	68.0	52.5	55.5	45.5	71.0	55.5	50.0	52.5	47.5	52.0	64.0	2.5
NLI	82.4	78.8	75.3	78.9	70.2	69.8	66.4	78.1	<u>75.7</u>	67.7	71.0	67.4	66.6	67.5	14.9
Overall	63.6	59.6	<u>53.7</u>	59.0	48.9	52.7	48.9	58.8	53.5	50.4	51.2	50.0	49.7	34.9	16.0

1) *Training And Evaluation Datasets*: We leverage a subset of flan-v2 datasets [5] as shown in Fig.9 for LoRA expert training and mixed-task dataset generation. We adopt the task cluster as described by [5]. The details of datasets can be found in the Appendix. B.

2) *Base Model & LoRA Configuration*.: To test various methods in the mixed-task scenarios, we leverage Llama-2-7b,13b [4] as base models and train a range of LoRAs for a spectrum of tasks. We selected a portion of the Flan-v2 datasets [5] to train 48 LoRAs for a spectrum of tasks covering Natural Language Understanding (NLU) and Natural Language Generation (NLG). Following the categorization by [5], these tasks can be grouped into 10 distinct task clusters. We train each LoRA according to the Alpaca [52] format and rank r , and the scaling hyperparameter α are set to 6 and 12, respectively.

3) *Evaluation Dataset*.: For constructing the evaluation dataset, we randomly chose 50 samples from the test set for each task used in training 48 LoRAs, subsequently mixing and shuffling these samples to form a unified dataset with 2400 data entries.

4) *Baseline Methods*.: We compared our method with the following baselines: (1) **Mixture of Experts** [11], [21], [22], [24], [33]. Many works have considered coordinating different adapters through MoE, and here we explored three distinct variants: one employing a soft mixture of experts and the

other utilizing discrete routing (top1 and top3). It is worth noting that in this paper, our primary focus is on post-hoc MoE training. This implies that the LoRA experts have been previously trained, and only the parameters of the router are trainable during the training of the MoE methods. (2) **SMEAR** [23] introduces the concept of adaptive routing by performing a weighted average of different adapters' parameters to utilize various experts effectively. For the MoE and SMEAR baselines, challenges arise in scaling due to training confined to a limited set of LoRAs. Consequently, we strategically selected a dedicated LoRA expert for each domain to specialize in router training. (3) **AdapterSoup** [26] uniformly selects the corresponding LoRAs for the entire downstream task, which lacks the ability to provide personalized service for diverse requests. (4) **LoRAHub** [25] enables black-box optimization to learn the weights of various LoRA parameters, thereby facilitating weighted parameter averaging for specific downstream tasks. In our implementation, we conformed to the default setting, which entails randomly selecting 20 LoRAs from the available LoRA pool and performing weighted parameter averaging. For the MoE, SMEAR, and LoRAHub approaches, we selected 20 data samples from the training datasets of all tasks to serve as their training data.

5) *Implementation of Baseline Methods*.: **MoE baselines**. We use E to denote the LoRA expert and R to denote the

Table II
COMPARISON OF SENTENCE EMBEDDING TECHNIQUES IN LORA RETRIEVAL: THE NOTATION LORARETRIEVER^{k%} SIGNIFIES THAT THE MODEL UNDERWENT SUPPLEMENTARY TRAINING ON k PERCENT OF THE TASKS. THE PERFORMANCE OF THE SELECTED RETRIEVER MODEL IN THE EVALUATION PHASE IS HIGHLIGHTED IN GRAY.

Method	Top 1	Top 3	Top 5	Top 8
all-mpnet-base-v2	58.40	78.26	84.77	90.24
all-MiniLM-L6-v2	51.73	73.11	80.54	87.18
msmarco-distilbert-cos-v5	45.84	66.01	75.14	82.67
gtr-t5-xl	53.19	69.72	77.41	83.59
LoraRetriever ^{0%}	60.80	79.29	85.57	91.58
LoraRetriever ^{40%}	63.16	89.09	95.45	98.97
LoraRetriever ^{100%}	74.08	97.37	99.15	99.82

router. The MoE methods can be expressed in the following way:

$$y = \sum_{i=1}^k R(x)_i E_i(x). \quad (16)$$

We implied two variants of the MoE routing mechanism. (1) **Dense Gating**. Following [22], the router network consists of a dense layer with trainable parameter W_g , and the gating score could be obtained through a softmax function by:

$$s_i = R(x)_i = \text{softmax}(W_g^T x), \quad (17)$$

(2) **Sparse Gate**. To maintain the sparsity while training, we leverage the Gumbel softmax trick as [23], [53], where the router can be written as:

$$\hat{R}(x)_i = \frac{(\log(R(x)_i) + g_i)/\tau}{\sum_{i=1}^k \exp((\log(R(x)_i) + g_i)/\tau)} \quad (18)$$

where $g_i \sim \text{Gumbel}(0, 1)$ and τ is the temperature.

Due to MoE not being easily scalable and arbitrarily adding new LoRAs, we randomly selected a LoRA as an expert for each task cluster in the experiment and trained the corresponding Router’s parameters. We randomly selected 20 samples for each task during training to form a unified dataset for parameter training.

SMEAR SMEAR [23] does not perform routing aggregation on the Adapter output but rather aggregates the Adapter at the parameter level. We adopt the same setting as the MoE methods, and the results could be calculated in the following way:

$$\Theta_{\text{SMEAR}} = \sum_{i=1}^k R(x)_i \Theta_i, \quad (19)$$

where Θ_i denote the parameter of the LoRA- i .

AdapterSoup AdapterSoup [26], for new downstream tasks, retrieves the parameters that need to be involved in aggregation through sentence bert and performs weight-space averaging on these parameters to adapt to the new domain. We have uniformly retrieved 3 LoRAs for mixed-task to test their capabilities under mixed-task conditions.

LoRAHub LoRAHub [25] also aggregates 20 LoRAs randomly for new downstream tasks. In order to learn the weight of LoRA, a black-box optimization method is employed to

learn the weight of each LoRA without calculating the gradients of the large model. It performs weighted averaging at the parameter level. Similar to the training process of MoE, we randomly selected 20 samples for each task to form a unified training dataset for black-box optimization.

6) *Implementation of RAMoLE.*: To train RAMoLE, we divide our training process into two stages: first, training the LoraRetriever, and second, training the RouterLoRA. To train the LoraRetriever, we continue to perform instruction fine-tuning based on Instructor-xl [27]. The training data consisted of only 40% of the tasks used to train task-specific LoRAs, with each task represented by 20 samples randomly selected from its respective LoRA training set. In this process, we categorized samples from the same LoRA as positive examples, while those from different LoRAs were considered negative examples.

For training the RouterLoRA, we also use only 40% of the tasks and the corresponding LoRAs for training, leaving the remaining LoRAs unseen to test the generalization of the on-the-fly MoE mechanism. During the training process, we load all these LoRAs into the LLM and randomly dropout 50% of the LoRAs to enhance the robustness of the router training. The RouterLoRA shares the same configuration as the task LoRA, with a rank of $r = 6$ and a scaling hyperparameter of $\alpha = 12$.

Additionally, we compared three vanilla LoRA composition strategies: (1) **Selection**, which involves selecting the highest-ranked (top-1) retrieved LoRA and applying it singularly, serving as a variant of the Mixture and Fusion methods; (2) **Mixture**, which averages the outputs of each submodule from the top- k retrieved LoRAs; and (3) **Fusion**, a method that averages the parameters of the top- k retrieved LoRAs. Throughout our experiments, $k = 3$ was established as the default setting.

7) *Metrics.*: Following [5], we assess the performance on the "Struct to Text" task using Rouge- $\{1, 2, L\}$ and on the "Translation" tasks using BLEU. Additionally, for the NLU tasks, we evaluate the exact match accuracy of each method.

A. Main Results

The main results of the mixed-task evaluation are shown in Tab.I. We present the mean performance across each task cluster. From the results, we have the following observations:

- The proposed framework, DAMoLE, which performs input-aware LoRA retrieval and on-the-fly MoLE, markedly surpasses other baselines focusing on specific downstream tasks.
- For the vanilla LoRA composition methods, the performance of Mixture and Selection is similar in IID scenarios, while Fusion’s performance is weaker compared to the other two methods. The reasons are as follows: (i) In the IID setting, LoraRetriever can achieve strong top-1 selection, leading to similar results between Selection and the Mixture; (ii) As different tasks are inherently heterogeneous, it is inferior to directly average top- k LoRA parameters in the Fusion. In the OOD setting, the Mixture exceeds the performance of the Selection,

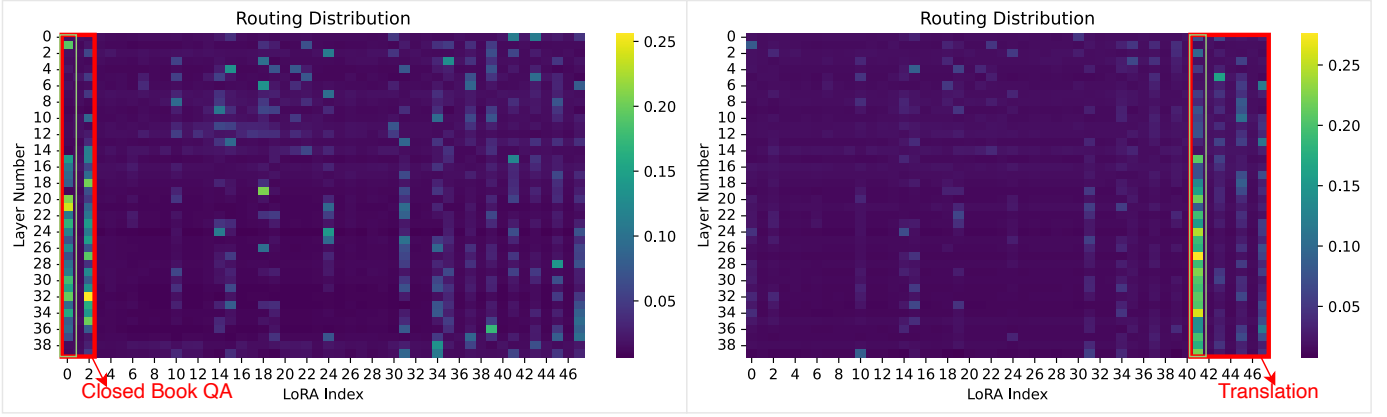


Figure 3. DAMoLE Routing Distribution for NQ (Closed-book QA) and WMT-16 De/En (Translation). Red square brackets denote LoRAs from the same task cluster and yellow square brackets highlight the ideal LoRA for the respective tasks.

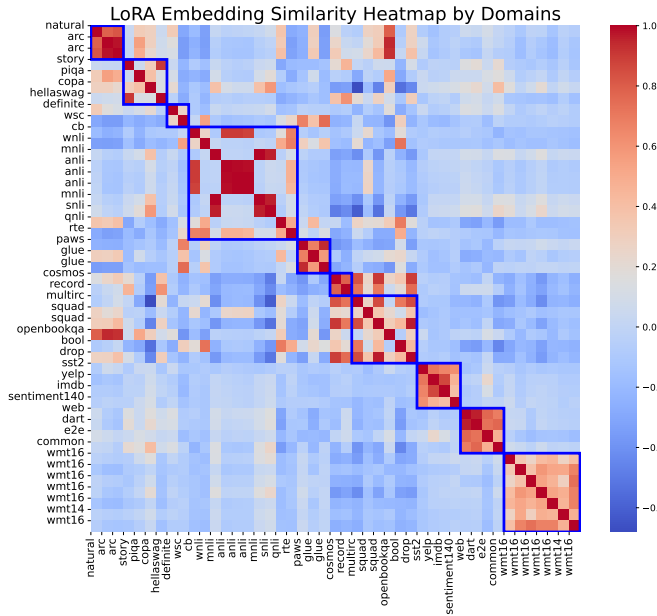


Figure 4. LoRA embedding similarity heatmap. Tasks from the same domain are grouped in square brackets.

and the performance of Fusion is similar to that of the Selection. The reasons can be as follows: (i) The selection cannot retrieve the associated LoRA for the input sample in the OOD setting, leading to a significant performance drop. (ii) The Mixture can fully leverage the capabilities of similar tasks to address OOD tasks, alleviating the performance drop.

- Compared to the vanilla LoRA composition strategies, DAMoLE demonstrates superior performance. (i) In the IID setting, DAMoLE outperforms the Selection method because it can route to the most suitable LoRA for each task. Additionally, DAMoLE retrieves more LoRA parameters in the initial stage, achieving better recall than the Selection method, which only selects the top-1 LoRA during the retrieval stage. (ii) In the OOD setting, DAMoLE also achieves superior performance,

particularly when compared with the Mixture method. This is because DAMoLE learns to coordinate different LoRAs and differentiates each LoRA during inference. In contrast, the Mixture method simply assigns uniform weights to each LoRA, which results in suboptimal performance.

- The performance of the MoE and SMEAR methods is weaker than that of DAMoLE. The limitation stems from the restricted capacity of these methods for adaptation and generalization to dynamically changing environments populated with diverse LoRAs, thereby diminishing their efficacy in mixed-task scenarios.
- In mixed-task scenarios, although AdapterSoup uniformly searches for appropriate LoRAs for downstream tasks, the retrieved LoRAs fall short in personalization for each request, hindering their effectiveness for each specific task.
- LoRAHub proves to be entirely ineffective in the mix-task scenario. First, the fusion of LoRAHub depends on randomly selected LoRAs, which may not be relevant. Second, the presence of heterogeneous tasks introduces conflicting parameter optimization directions, resulting in the total breakdown of parameter fusion.

B. Analysis

a) Performance of Retriever. We compare LoraRetriever with some popular off-the-shelf sentence embedding models in Huggingface and adopt the model nomenclature following [54]. To analyze the effect of the percentage of tasks for training LoraRetriever, we trained three variants of LoraRetriever with different percentages. Tab.II shows the performance of different retrieval models for retrieving relevant LoRAs. It is shown that guiding sentence embedding models with specific prompts leads to a performance improvement in retrieval compared to common retrieval models. After instruction fine-tuning, the retriever significantly enhanced the ability to retrieve the corresponding LoRA based on the input. Conducting instruction fine-tuning on 40% of the tasks resulted in a 2.36% increase in top-1 accuracy and a 9.80% increase in top-3 accuracy. Training across all tasks achieved the largest

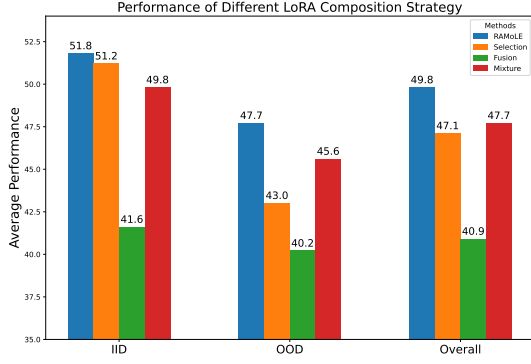


Figure 5. Comparison of Different LoRA Composition Strategies. The overall performance averages the IID and OOD results to demonstrate the comprehensive effectiveness of various LoRA composition strategies.

improvement. To demonstrate the generalizability of the proposed framework when dealing with unseen LoRAs, we used a retriever trained on 40% of the tasks in the main experiment to simulate the scenario of dynamic updates to the LoRA pool that might occur while providing services with LoraRetriever.

Fig.4 illustrates the similarity between task embeddings for different tasks through a heatmap, where tasks from the same task cluster are grouped in square brackets. It is shown that task embeddings within the same domain are more similar, indicating that the LoraRetriever embeddings can serve as task embeddings to characterize the similarities between different tasks and can be applied for LoRA retrieval.

b) Performance of the On-the-fly MoLE Mechanism:

Fig. 3 illustrates the routing distribution performance of the proposed mechanism across two tasks: Closed Book QA and Translation. The figure uses red square brackets to denote LoRAs from the same task cluster and yellow square brackets to highlight the ideal LoRA for the respective tasks. The results demonstrate that the proposed MoLE mechanism effectively assigns a higher weight to the ideal LoRA for each input at the module level. Furthermore, the importance of higher-level layers in the routing process is evident, which corroborates the findings of [55].

In Fig.5, we compare the performance of different LoRA composition strategies across various settings. The results reveal that: (i) Among vanilla LoRA composition methods, the Selection strategy performs best in the IID setting as it effectively utilizes the ideal LoRA retrieved from LoraRetriever. The Mixture strategy excels in the OOD setting due to its ability to leverage multiple similar-task LoRAs, enhancing its generalization capability. (ii) RAMoLE achieves the best performance in both IID and OOD settings; in the IID context, it capitalizes on the optimal LoRA’s strengths to compensate for the shortcomings of the Mixture strategy, while in the OOD context, it coordinates the capabilities of different LoRAs to address an unseen task effectively. Overall, RAMoLE can balance the accuracy of LoRA selection and the coordination between them, thereby achieving optimal results.

We also conducted ablation experiments to demonstrate the effectiveness of the LoRA Dropout strategy during RouterLoRA training. The results are depicted in Fig.6. In the IID

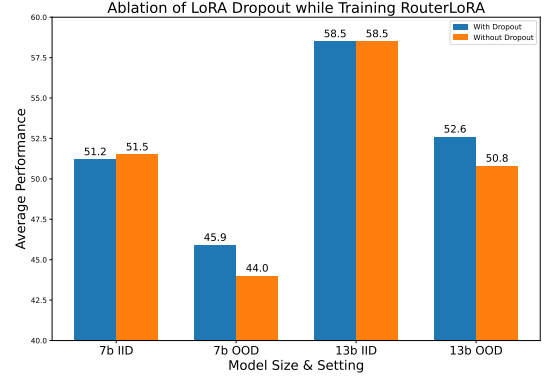


Figure 6. Ablation of LoRA Dropout Strategy during RouterLoRA training.

setting, since all training LoRAs are loaded into the LLM during training, it is easier to learn the pattern of routing to the ideal LoRA during inference. Consequently, even without LoRA Dropout, satisfactory performance is achieved in the IID setting. However, in the OOD setting, where the ideal LoRA for each sample is blocked, RouterLoRA must learn to coordinate different LoRAs to solve an unseen task. In this scenario, training without LoRA Dropout proves ineffective, and we observe a significant increase when performing LoRA Dropout in the OOD setting.

c) Impact of the number of Retrieved LoRA: Fig.7 (a) illustrates the performance of the number of retrieved LoRAs on the mean accuracy of the NLU tasks. The results indicate that as the number of retrieved LoRAs increases, the performance of the DAMoLE and vanilla Mixture initially improves slightly but then stabilizes, while DAMoLE outperforms other methods in all settings. In contrast, the Fusion shows a continuous decline in performance with an increasing number of LoRAs, which once again demonstrates that under the conditions of heterogeneous tasks, the simple averaging of parameters can compromise the original capabilities of the LoRAs. DAMoLE consistently outperforms other methods across all settings. In particular, in the OOD setting, the performance of the DAMoLE and Mixture improves significantly as the number of LoRAs increases, illustrating that in the absence of an ideal LoRA choice for a request, leveraging the capabilities of multiple LoRAs of similar tasks can effectively achieve cross-task generalization.

d) Effectiveness of Batch Inference Strategy.: To evaluate the efficiency of our proposed batch inference strategy, we compared the throughput of different batch sizes. Throughput is defined as the number of both input and output tokens per second across all requests in the mixed-task benchmark. We specifically compared the computational efficiency with that of a single LoRA. Our evaluation encompassed the entire evaluation dataset, and we limited the generation to the first produced token to mitigate discrepancies caused by varying generation lengths across different methods. These experiments were carried out on an NVIDIA A100 GPU (80GB) utilizing bfloat16 precision. As illustrated in Fig.7 (b), our batch inference strategy markedly improves the throughput of the framework, with a slight throughput reduction compared

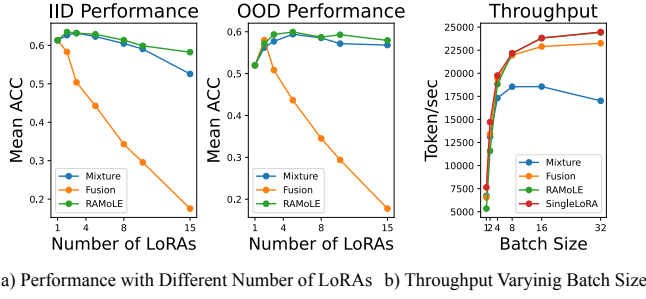


Figure 7. The left figure shows the performance of LoraRetriever varying the number of LoRAs. The right figure shows the performance of Throughput varying the batch size.

to a single LoRA. Notably, the Fusion strategy outperforms the mixture strategy in throughput efficiency, attributed to its parameter averaging approach, which circumvents the need for parallel computation across multiple LoRAs. RAMoLE outperforms other composition methods by first gathering the corresponding LoRA for each input, allowing for a more effective inference process. As the batch size increases, the throughput of RAMoLE gradually reaches parity with that of a single LoRA.

e) Showcases.: We showcase the framework’s ability to adeptly integrate multiple LoRAs for synergistic problem solving, as evidenced in Fig.8. We manually craft three problems in Fig.8, which cannot retrieve any single LoRA to solve these problems directly, necessitating the cooperation of existing LoRAs. Specifically, the first example requires RAMoLE to integrate NLI and translation tasks’ capabilities. The retrieved LoRA wmt16-tren is utilized for comprehending Turkish, while glue-qqp is applied to NLI tasks. In the second scenario, LoRAs are integrated for translating from German to French. Although there is no direct LoRA for German-to-French translation, the combined use of wmt16-deen for German-to-English and wmt14-enfr for English-to-French enables an effective German-to-French translation. The third scenario illustrates the fusion of distinct capabilities by combining Romanian translation with text generation: leveraging the wmt16-roen LoRA for Romanian comprehension and the common-gen LoRA for generating text, RAMoLE successfully merges these diverse functionalities. This demonstration emphasizes the framework’s substantial ability to blend distinct LoRA capabilities, anticipating further exploration of capability fusion of LoRAs as a future direction.

VI. LIMITATIONS AND POTENTIAL FUTURE DIRECTIONS

While promising, there are still some drawbacks of RAMoLE. (1) User data privacy issues. When users upload LoRA, we need to use a small amount of training data (10-20 pieces) to represent the distribution of the LoRA model. In privacy-sensitive scenarios, representation with data may not be feasible. Aligning LoRA parameters and sample distributions in the embedding space in a manner that respects data privacy presents a worthwhile direction for future exploration. (2) The proposed RAMoLE framework is only suitable for multi-LoRA collaboration under the same model architecture.

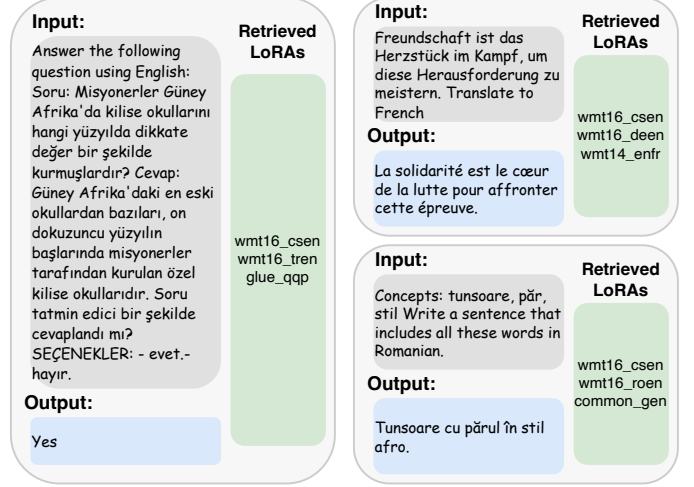


Figure 8. Showcasing How the RAMoLE Framework Employs Multiple LoRAs for Cooperative Problem Solving.

However, in reality, the model architecture chosen by the users themselves and the PEFT method are not necessarily the same, which is worth further research on how to design the corresponding collaborative mechanism for such scenarios.

VII. CONCLUSION

In this paper, we explore a new and increasingly significant computing paradigm, *Uploadable Machine Learning*. The challenge in this scenario is to leverage a dynamically changing and continuously updated LoRA pool to deliver personalized services for heterogeneous downstream requests. To this end, we introduce a novel framework called RAMoLE which first identifies and retrieves suitable LoRAs based on specific prompts and then performs on-the-fly LoRA composition. The composition process incorporates a novel MoLE mechanism that uses an off-the-shelf RouterLoRA to assign weights to each LoRA. Additionally, we have developed an efficient batch inference strategy to handle batched requests. Further experiments have demonstrated the effectiveness of our proposed RAMoLE framework.

REFERENCES

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [4] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [5] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.
- [6] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

- [7] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [8] A. Young, B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, H. Li, J. Zhu, J. Chen, J. Chang *et al.*, “Yi: Open foundation models by 01. ai,” *arXiv preprint arXiv:2403.04652*, 2024.
- [9] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili *et al.*, “A survey on large language models: Applications, challenges, limitations, and practical usage,” *Authorea Preprints*, 2023.
- [10] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2023.
- [11] Q. Liu, X. Wu, X. Zhao, Y. Zhu, D. Xu, F. Tian, and Y. Zheng, “Moelora: An moe-based parameter efficient fine-tuning method for multi-task medical applications,” *arXiv preprint arXiv:2310.18339*, 2023.
- [12] H. Yang, X.-Y. Liu, and C. D. Wang, “Fingpt: Open-source financial large language models,” *arXiv preprint arXiv:2306.06031*, 2023.
- [13] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang, “P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022, pp. 61–68.
- [14] Z. Hu, Y. Lan, L. Wang, W. Xu, E.-P. Lim, R. K.-W. Lee, L. Bing, and S. Poria, “Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models,” *arXiv preprint arXiv:2304.01933*, 2023.
- [15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [16] M. Luo, J. Wong, B. Trabucco, Y. Huang, J. E. Gonzalez, Z. Chen, R. Salakhutdinov, and I. Stoica, “Stylus: Automatic adapter selection for diffusion models,” *arXiv preprint arXiv:2404.18928*, 2024.
- [17] Z. Zhao, L. Gan, G. Wang, W. Zhou, H. Yang, K. Kuang, and F. Wu, “Loraretriever: Input-aware lora retrieval and composition for mixed tasks in the wild,” *arXiv preprint arXiv:2402.09997*, 2024.
- [18] H. Liu, Z. Liu, R. Tang, J. Yuan, S. Zhong, Y.-N. Chuang, L. Li, R. Chen, and X. Hu, “Lora-as-an-attack! piercing llm safety under the share-and-play scenario,” *arXiv preprint arXiv:2403.00108*, 2024.
- [19] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [20] M. I. Jordan and R. A. Jacobs, “Hierarchical mixtures of experts and the em algorithm,” *Neural computation*, vol. 6, no. 2, pp. 181–214, 1994.
- [21] Y. Wang, S. Agarwal, S. Mukherjee, X. Liu, J. Gao, A. H. Awadallah, and J. Gao, “Adamix: Mixture-of-adaptations for parameter-efficient model tuning,” *arXiv preprint arXiv:2210.17451*, 2022.
- [22] T. Zadouri, A. Üstün, A. Ahmadian, B. Ermiş, A. Locatelli, and S. Hooker, “Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning,” *arXiv preprint arXiv:2309.05444*, 2023.
- [23] M. Muqeeth, H. Liu, and C. Raffel, “Soft merging of experts with adaptive routing,” *arXiv preprint arXiv:2306.03745*, 2023.
- [24] X. Wu, S. Huang, and F. Wei, “Mole: Mixture of lora experts,” in *The Twelfth International Conference on Learning Representations*, 2023.
- [25] C. Huang, Q. Liu, B. Y. Lin, T. Pang, C. Du, and M. Lin, “Lorahub: Efficient cross-task generalization via dynamic lora composition,” 2023.
- [26] A. Chronopoulou, M. E. Peters, A. Fraser, and J. Dodge, “Adaptersoup: Weight averaging to improve generalization of pretrained language models,” *arXiv preprint arXiv:2302.07027*, 2023.
- [27] H. Su, W. Shi, J. Kasai, Y. Wang, Y. Hu, M. Ostendorf, W.-t. Yih, N. A. Smith, L. Zettlemoyer, and T. Yu, “One embedder, any task: Instruction-finetuned text embeddings,” *arXiv preprint arXiv:2212.09741*, 2022.
- [28] A. Asai, T. Schick, P. Lewis, X. Chen, G. Izacard, S. Riedel, H. Hajishirzi, and W.-t. Yih, “Task-aware retrieval with instructions,” *arXiv preprint arXiv:2211.09260*, 2022.
- [29] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei, “Bert loses patience: Fast and robust inference with early exit,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 18 330–18 341. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/file/d4dd111a4fd973394238aca5c05bebe3-Paper.pdf>
- [30] S. Shen, L. Hou, Y. Zhou, N. Du, S. Longpre, J. Wei, H. W. Chung, B. Zoph, W. Fedus, X. Chen *et al.*, “Mixture-of-experts meets instruction tuning: A winning combination for large language models,” *arXiv preprint arXiv:2305.14705*, 2023.
- [31] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, “Scaling vision with sparse mixture of experts,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.
- [32] S. Dou, E. Zhou, Y. Liu, S. Gao, J. Zhao, W. Shen, Y. Zhou, Z. Xi, X. Wang, X. Fan, S. Pu, J. Zhu, R. Zheng, T. Gui, Q. Zhang, and X. Huang, “Loramoe: Revolutionizing mixture of experts for maintaining world knowledge in language model alignment,” 2023.
- [33] Y. Zhu, N. Wichers, C.-C. Lin, X. Wang, T. Chen, L. Shu, H. Lu, C. Liu, L. Luo, J. Chen *et al.*, “Sira: Sparse mixture of low rank adaptation,” *arXiv preprint arXiv:2311.09179*, 2023.
- [34] O. Ostapenko, L. Caccia, Z. Su, N. Le Roux, L. Charlin, and A. Sordoni, “A case study of instruction tuning with mixture of parameter-efficient experts,” in *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023.
- [35] W. Feng, C. Hao, Y. Zhang, Y. Han, and H. Wang, “Mixture-of-loras: An efficient multitask tuning for large language models,” *arXiv preprint arXiv:2403.03432*, 2024.
- [36] M. Muqeeth, H. Liu, Y. Liu, and C. Raffel, “Learning to route among specialized experts for zero-shot generalization,” *arXiv preprint arXiv:2402.05859*, 2024.
- [37] O. Ostapenko, Z. Su, E. M. Ponti, L. Charlin, N. L. Roux, M. Pereira, L. Caccia, and A. Sordoni, “Towards modular llms by building and reusing a library of loras,” *arXiv preprint arXiv:2405.11157*, 2024.
- [38] J. Zhang, S. Chen, J. Liu, and J. He, “Composing parameter-efficient modules with arithmetic operations,” *arXiv preprint arXiv:2306.14870*, 2023.
- [39] H. Wang, B. Ping, S. Wang, X. Han, Y. Chen, Z. Liu, and M. Sun, “Lora-flow: Dynamic lora fusion for large language models in generative tasks,” *arXiv preprint arXiv:2402.11455*, 2024.
- [40] C. Wu, T. Wang, Y. Ge, Z. Lu, R. Zhou, Y. Shan, and P. Luo, “ π -tuning: Transferring multimodal foundation models with optimal multi-task interpolation,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 37 713–37 727.
- [41] Y. Yang, W. Wang, L. Peng, C. Song, Y. Chen, H. Li, X. Yang, Q. Lu, D. Cai, B. Wu *et al.*, “Lora-composer: Leveraging low-rank adaptation for multi-concept customization in training-free diffusion models,” *arXiv preprint arXiv:2403.11627*, 2024.
- [42] M. Zhong, Y. Shen, S. Wang, Y. Lu, Y. Jiao, S. Ouyang, D. Yu, J. Han, and W. Chen, “Multi-lora composition for image generation,” *arXiv preprint arXiv:2402.16843*, 2024.
- [43] A. Tang, L. Shen, Y. Luo, N. Yin, L. Zhang, and D. Tao, “Merging multi-task models via weight-ensembling mixture of experts,” *arXiv preprint arXiv:2402.00433*, 2024.
- [44] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer *et al.*, “S-lora: Serving thousands of concurrent lora adapters,” *arXiv preprint arXiv:2311.03285*, 2023.
- [45] Y. Wen and S. Chaudhuri, “Batched low-rank adaptation of foundation models,” *arXiv preprint arXiv:2312.05677*, 2023.
- [46] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv:1908.10084*, 2019.
- [47] J. Ni, G. H. Abrego, N. Constant, J. Ma, K. B. Hall, D. Cer, and Y. Yang, “Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models,” *arXiv preprint arXiv:2108.08877*, 2021.
- [48] A. Achille, M. Lam, R. Tewari, A. Ravichandran, S. Maji, C. C. Fowlkes, S. Soatto, and P. Perona, “Task2vec: Task embedding for meta-learning,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6430–6439.
- [49] W. Zhou, C. Xu, and J. McAuley, “Efficiently tuned parameters are task embeddings,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 5007–5014. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.334>
- [50] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” *arXiv preprint arXiv:2004.04906*, 2020.
- [51] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised dense information retrieval with contrastive learning,” *arXiv preprint arXiv:2112.09118*, 2021.
- [52] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model,” https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [53] X. Nie, S. Cao, X. Miao, L. Ma, J. Xue, Y. Miao, Z. Yang, Z. Yang, and C. Bin, “Dense-to-sparse gate for mixture-of-experts,” 2021.

- [54] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [55] C. Gao, K. Chen, J. Rao, B. Sun, R. Liu, D. Peng, Y. Zhang, X. Guo, J. Yang, and V. Subrahmanian, “Higher layers need more lora experts,” *arXiv preprint arXiv:2402.08562*, 2024.



Ziyu Zhao received his B.S. degree in 2021 from the Department of Computer Science and Technology, Zhejiang University. He is a third-year Ph.D. candidate in the Department of Computer Science and Technology at Zhejiang University. His main research interests include Large Language Models, machine learning, and data mining.



Leilei Gan is currently an assistant professor in the College of Software and Technology at Zhejiang University. Before that, He obtained his Ph.D. degree in computer science and technology from Zhejiang University. His research interests include natural language processing and deep learning. Currently, he studies foundation models, including pre-training, fine-tuning, alignment, and their applications on downstream tasks.



Guoyin Wang Dr. Guoyin Wang obtained his B.S. degree from Peking University in 2011, followed by an M.S. degree from the University of Illinois at Urbana-Champaign in 2013, and a Ph.D. from Duke University in 2020. He has authored over 50 papers in the fields of deep learning and natural language processing. His primary research interests encompass large language models, machine learning, and natural language processing. He has also served as a reviewer and area chair for prestigious academic conferences, including ACL, EMNLP, NeurIPS, and

ICML.



Yuwei Hu Yuwei Hu is a junior undergraduate student at the College of Computer Science and Technology at Zhejiang University. He is deeply engaged in the field of computer science, with a strong focus on cutting-edge technologies including large language models, machine learning, and reinforcement learning for intelligent agents.



Tao Shen Tao Shen received his BS degree in control science and technology from China University of Petroleum, Qingdao, China, in 2015. He is currently pursuing his Ph.D. in computer science and technology at Zhejiang University. His research interests focus on federated learning.



Hongxia Yang Dr. Hongxia Yang, PhD from Duke University, led the team to develop AI open sourced platforms and systems such as AliGraph, M6, Luoxi. Dr. Yang has published over 100 top conference and journal papers, and held more than 50 patents. She has been awarded the highest prize of the 2019 World Artificial Intelligence Conference, Super AI Leader (SAIL Award), the second prize of the 2020 National Science and Technology Progress Award, the first prize of Science and Technology Progress of the Chinese Institute of Electronics in 2021, the

Forbes China Top 50 Women in Science and Technology and Ministry of Education Science and Technology Progress Award First Class in 2022 and AI 2000 Most Influential Scholar Award in 2023-2024. She used to work as the Senior Staff Data Scientist and Director in Alibaba Group, Principal Data Scientist at Yahoo! Inc and Research Staff Member at IBM T.J. Watson Research Center, joint adjunct professor at Zhejiang University Shanghai Advanced Research Institute respectively, Bytedance US Head.



Fei Wu received his Ph.D. degree from the College of Computer Science at Zhejiang University. Now he is the professor and dean of the College of Computer Science, Zhejiang University. His main research interests include multimedia information analysis and retrieval, and digital library.



Kun Kuang received his Ph.D. degree from Tsinghua University in 2019. He is now an Associate Professor in the College of Computer Science and Technology, Zhejiang University. He was a visiting scholar with Prof. Susan Athey's Group at Stanford University. His main research interests include Causal Inference, Artificial Intelligence, and Causally Regularized Machine Learning. He has published over 60 papers in major international journals and conferences, including SIGKDD, ICML, NeurIPS, ACM MM, AAAI, TKDE, TKDD, Engi-

neering, and ICDM, etc.

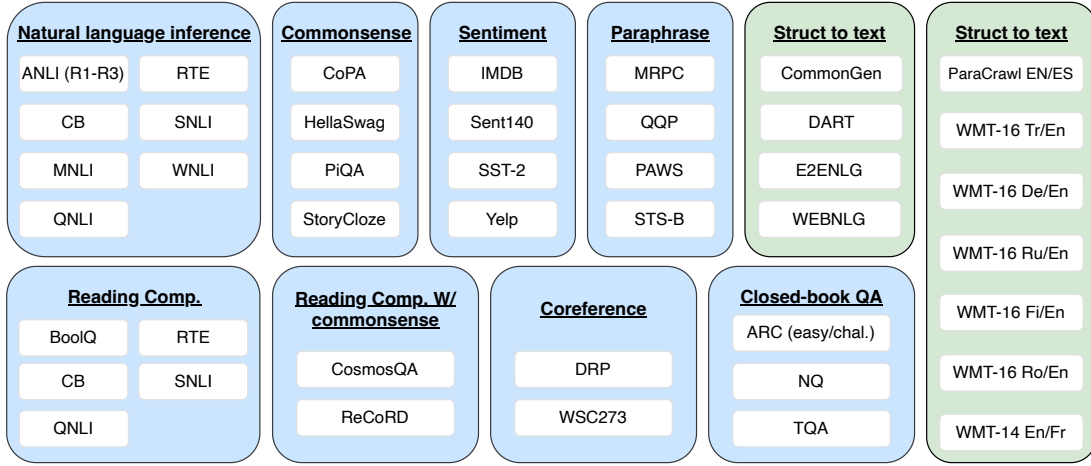


Figure 9. Datasets and task clusters used to train LoRAs and generate mixed-task evaluation set in this paper (NLU tasks in blue; NLG tasks in green).

APPENDIX A FULL RESULTS

The full results on the evaluation dataset are shown in Tab.III & Tab.IV.

APPENDIX B DETAILS OF TRAINING AND EVALUATION DATASETS

We leverage a subset of flan-v2 datasets [5] as shown in Fig.9 for LoRA expert training and mixed-task dataset generation. We summarize the details of the used datasets as follows:

Struct-to-Text Conversion: This task evaluates the capability to generate natural language descriptions from structured data inputs. We use the following datasets: (1) CommonGen; (2) DART; (3) E2ENLG; (4) WebNLG;

Translation: Translation involves converting text from one language to another, maintaining the original meaning and nuances. We use the following datasets: (1) En-Fr from WMT’14; En-De, En-Tr, En-Ru, En-Fi, En-Ro from WMT’16; (3) En-Es from Paracrawl.

Commonsense Reasoning: This involves assessing the ability to apply physical or scientific principles alongside common sense in reasoning tasks. We use the following datasets: (1) COPA, (2) HellaSwag, (3) PiQA, and (4) StoryCloze.

Sentiment Analysis: A fundamental task in natural language processing (NLP) that determines the sentiment polarity (positive or negative) of a given text. We use the following datasets: (1) IMDB, (2) Sentiment140, (3) SST-2, and (4) Yelp.

Closed-Book Question Answering: This task challenges models to answer questions about general knowledge without direct access to external information sources. We use the following datasets: (1) ARC, (2) NQ, and (3) TriviaQA.

Paraphrase Detection: This task requires models to ascertain whether two sentences convey the same meaning, indicating semantic equivalence. We use the following datasets: (1) MRPC, (2) QQP, and (3) Paws Wiki.

Coreference Resolution: Involves identifying instances within a text that refer to the same entity, demonstrating

an understanding of textual context. We use the following datasets: (1) DPR and (2) WSC273.

Reading comprehension: Assesses the capability to derive answers to questions from a provided text containing relevant information. We use the following datasets: (1) BoolQ, (2) DROP, (3) MultiRC, (4) OBQA, (5) SQuADv1, (6) SQuADv2.

Reading Comprehension with Commonsense: Merges traditional reading comprehension skills with commonsense reasoning, requiring understanding beyond the explicit text. We use the following datasets: (1) CosmosQA; (2) ReCoRD.

Natural Language Inference: Focuses on deducing the relationship between two sentences, determining if the second sentence logically follows from, contradicts, or is unrelated to the first sentence. We use the following datasets: (1) ANLI, (2) CB; (3) MNLI; (4) QNLI; (5) SNLI; (6) WNLI; (7) RTE.

Table III
MIXED TASKS EVALUATION ON BOTH NLU & NLG TASKS. “OOD” INDICATES THAT DURING RETRIEVAL, WE MASKED THE CORRESPONDING TASK’S
LoRA FOR TESTING GENERALIZATION WHEN FACING UNKNOWN TASKS.

Task	Perfect Selection	RAMoLE		Selection		Fusion		Mixture		MoE Top1	MoE Top3	MoE Soft	SME-AR	Adapter Soup	LoRA Hub
		IID	OOD	IID	OOD	IID	OOD	IID	OOD						
Struct to Text															
WebNLG Rouge-1	68.2	51.9	46.8	64.2	50.5	45.8	41.3	54.6	50.1	41.9	43.2	45.0	46.7	3.1	29.6
WebNLG Rouge-2	47.6	25.1	23.5	41.6	26.3	23.2	21.1	30.1	25.9	20.1	22.2	22.9	25.2	0.9	14.7
WebNLG Rouge-1	56.6	40.7	36.8	51.6	39.7	36.3	32.3	44.6	40.9	32.9	33.0	35.1	36.0	2.7	23.9
DART Rouge-1	66.6	65.8	50.6	63.1	52.0	51.1	48.8	58.2	54.8	51.7	52.4	52.5	56.0	1.9	36.7
DART Rouge-2	45.3	43.1	29.7	42.1	30.5	28.5	26.9	32.7	31.4	28.5	28.6	28.1	30.3	0.5	18.2
DART Rouge-1	55.1	54.3	39.9	52.7	42.6	39.9	37.6	46.7	43.8	41.8	42.1	41.3	45.1	1.7	29.0
E2ENLG Rouge-1	59.2	58.5	56.0	58.8	53.2	54.7	50.4	59.1	52.3	46.8	47.7	49.3	47.3	4.0	43.7
E2ENLG Rouge-2	34.3	32.9	30.6	33.6	29.6	29.8	27.1	33.2	27.6	23.0	23.6	25.0	23.4	2.3	21.9
E2ENLG Rouge-1	44.8	44.1	41.3	44.4	38.6	39.8	37.7	43.9	37.1	34.5	35.2	37.0	34.7	3.9	32.0
CommonGen Rouge-1	42.4	42.9	27.5	41.1	25.1	26.4	23.3	32.8	24.0	24.7	24.0	25.9	23.3	5.2	17.6
CommonGen Rouge-2	17.2	18.7	8.6	16.9	6.3	8.8	5.6	9.3	6.6	5.5	5.0	6.9	6.4	0.0	5.6
CommonGen Rouge-1	37.8	39.6	24.1	37.0	20.4	21.9	19.1	28.7	20.4	21.1	21.0	21.9	19.8	4.9	14.8
Translation															
WMT'16-tren	3.2	2.7	2.0	3.1	2.6	3.5	3.3	3.7	2.6	3.5	3.2	3.4	3.2	0.0	2.0
WMT'16-deen	18.9	18.4	18.4	18.7	20.3	17.9	18.8	18.8	18.7	11.6	14.0	14.7	16.6	1.1	11.4
WMT'16-ruen	10.8	9.9	10.3	10.4	9.8	9.2	9.3	11.0	10.8	6.2	7.8	8.3	7.3	0.0	4.8
WMT'16-fien	6.5	7.6	7.3	6.5	7.0	7.2	7.1	7.3	7.8	6.2	6.2	6.1	6.5	0.7	4.3
WMT'16-roen	13.9	15.9	12.9	14.0	12.3	12.8	13.3	13.1	12.2	9.8	10.7	10.1	10.3	0.3	8.0
WMT'14-enfr	16.5	16.2	17.1	16.1	16.9	17.7	18.0	17.8	18.0	15.9	17.3	17.1	16.4	3.5	15.2
WMT'16-csen	10.7	7.0	5.2	9.4	7.0	6.1	6.2	8.3	5.8	4.7	6.3	6.3	6.3	0.8	6.1
Paracrawl-enes	24.3	23.2	21.3	24.2	20.3	22.9	22.3	22.8	22.1	18.0	18.8	19.5	21.6	4.5	16.4
COMMONSENSE															
StoryCloze	72.0	78.0	90.0	62.0	42.0	72.0	68.0	84.0	58.0	74.0	70.0	70.0	68.0	62.0	48.0
PIQA	46.0	44.0	38.0	46.0	32.0	34.0	36.0	38.0	34.0	40.0	38.0	38.0	36.0	38.0	0.0
COPA	86.0	62.0	58.0	74.0	68.0	78.0	70.0	80.0	68.0	72.0	70.0	72.0	70.0	56.0	22.0
HellaSwag	46.0	36.0	32.0	40.0	42.0	20.0	18.0	44.0	40.0	32.0	30.0	26.0	26.0	28.0	0.0
sentiment															
SST-2	98.0	96.0	96.0	98.0	96.0	74.0	78.0	96.0	94.0	56.0	68.0	66.0	66.0	74.0	0.0
Yelp	98.0	98.0	98.0	94.0	94.0	96.0	96.0	98.0	98.0	86.0	90.0	86.0	84.0	80.0	0.0
IMDB	96.0	98.0	100.0	96.0	96.0	92.0	82.0	96.0	96.0	76.0	80.0	80.0	84.0	80.0	0.0
sentiment140	68.0	70.0	74.0	70.0	70.0	54.0	58.0	68.0	74.0	62.0	62.0	66.0	62.0	60.0	2.0
READING Comp.															
MultiRC	68.0	64.0	60.0	52.0	38.0	44.0	44.0	48.0	44.0	54.0	52.0	50.0	48.0	40.0	6.0
SQuADv2	62.0	26.0	8.0	56.0	12.0	30.0	20.0	22.0	16.0	24.0	24.0	26.0	22.0	16.0	0.0
SQuADv1	68.0	62.0	62.0	66.0	68.0	64.0	64.0	62.0	68.0	68.0	70.0	66.0	66.0	54.0	4.0
OBQA	82.0	78.0	68.0	68.0	58.0	64.0	60.0	78.0	66.0	62.0	64.0	66.0	60.0	40.0	0.0
BoolQ	84.0	82.0	74.0	60.0	60.0	68.0	70.0	80.0	76.0	74.0	68.0	76.0	70.0	72.0	6.0
drop	40.0	18.0	16.0	8.0	6.0	14.0	12.0	18.0	14.0	10.0	8.0	8.0	8.0	22.0	0.0
CLOSE-BOOK QA															
NQ	18.0	12.0	10.0	16.0	10.0	16.0	14.0	16.0	10.0	12.0	12.0	12.0	4.0	12.0	0.0
ARC-e	50.0	74.0	80.0	56.0	70.0	54.0	56.0	66.0	82.0	58.0	58.0	60.0	58.0	48.0	0.0
ARC-c	46.0	46.0	48.0	42.0	46.0	34.0	34.0	50.0	46.0	46.0	42.0	42.0	42.0	24.0	0.0
TriviaQa	66.0	56.0	56.0	46.0	46.0	60.0	46.0	48.0	56.0	46.0	42.0	46.0	24.0	42.0	4.0
COREFERENCE															
DPR	54.0	64.0	46.0	50.0	50.0	56.0	60.0	68.0	56.0	64.0	60.0	62.0	62.0	46.0	2.0
WSC	50.0	50.0	56.0	50.0	42.0	38.0	46.0	58.0	42.0	58.0	58.0	52.0	54.0	40.0	0.0
READ. COOMP. W/ COMMONSENSE															
CosmosQa	68.0	74.0	58.0	68.0	34.0	46.0	32.0	50.0	46.0	44.0	46.0	44.0	38.0	14.0	6.0
record	70.0	54.0	30.0	70.0	26.0	24.0	6.0	42.0	34.0	18.0	12.0	14.0	8.0	14.0	0.0
PARAPHRASE															
Paws Wiki	90.0	54.0	44.0	64.0	40.0	44.0	42.0	56.0	46.0	56.0	50.0	48.0	54.0	60.0	2.0
QQP	74.0	88.0	64.0	74.0	68.0	66.0	60.0	80.0	58.0	50.0	40.0	36.0	28.0	54.0	0.0
MRPC	60.0	62.0	60.0	58.0	58.0	60.0	62.0	60.0	58.0	42.0	44.0	40.0	42.0	60.0	2.0
STSB	38.0	34.0	8.0	36.0	16.0	12.0	12.0	30.0	20.0	20.0	20.0	20.0	14.0	12.0	0.0
NLI															
CB	88.9	82.2	66.7	80.0	62.2	77.8	57.8	86.7	66.7	68.9	64.4	68.9	62.2	55.6	13.3
WNLI	70.0	52.0	42.0	68.0	46.0	44.0	50.0	60.0	54.0	56.0	56.0	42.0	44.0	52.0	0.0
ANLI-r1	50.0	52.0	48.0	50.0	50.0	40.0	42.0	40.0	42.0	40.0	40.0	36.0	38.0	38.0	24.0
ANLI-r2	46.0	48.0	46.0	46.0	46.0	32.0	36.0	46.0	46.0	40.0	36.0	38.0	32.0	46.0	20.0
ANLI-r3	46.0	50.0	38.0	42.0	38.0	38.0	40.0	44.0	50.0	28.0	32.0	34.0	38.0	40.0	24.0
MNLI-m	88.0	76.0	84.0	84.0	88.0	62.0	66.0	80.0	88.0	48.0	54.0	50.0	56.0	76.0	0.0
MNLI-mm	92.0	84.0	90.0	90.0	94.0	64.0	82.0	88.0	90.0	48.0	48.0	50.0	60.0	84.0	2.0
SNLI	96.0	88.0	86.0	84.0	84.0	56.0	58.0	90.0	92.0	54.0	52.0	54.0	54.0	82.0	0.0
QNLI	94.0	66.0	62.0	94.0	26.0	46.0	48.0	74.0	38.0	56.0	56.0	54.0	60.0	70.0	0.0
RTE	52.0	68.0	64.0	62.0	72.0	54.0	58.0	70.0	76.0	64.0	58.0	56.0	64.0	80.0	22.0

Table IV
MIXED TASKS EVALUATION ON BOTH NLU & NLG TASKS. “OOD” INDICATES THAT DURING RETRIEVAL, WE MASKED THE CORRESPONDING TASK’S
LoRA FOR TESTING GENERALIZATION WHEN FACING UNKNOWN TASKS.

Task	Perfect Selection	RAMoLE		Selection		Fusion		Mixture		MoE Top1	MoE Top3	MoE Soft	SME-AR	Adapter Soup	LoRA Hub
		IID	OOD	IID	OOD	IID	OOD	IID	OOD						
Struct to Text															
WebNLG Rouge-1	68.2	51.9	46.8	64.2	50.5	45.8	41.3	54.6	50.1	41.9	43.2	45.0	46.7	3.1	29.6
WebNLG Rouge-2	47.6	25.1	23.5	41.6	26.3	23.2	21.1	30.1	25.9	20.1	22.2	22.9	25.2	0.9	14.7
WebNLG Rouge-1	56.6	40.7	36.8	51.6	39.7	36.3	32.3	44.6	40.9	32.9	33.0	35.1	36.0	2.7	23.9
DART Rouge-1	66.6	65.8	50.6	63.1	52.0	51.1	48.8	58.2	54.8	51.7	52.4	52.5	56.0	1.9	36.7
DART Rouge-2	45.3	43.1	29.7	42.1	30.5	28.5	26.9	32.7	31.4	28.5	28.6	28.1	30.3	0.5	18.2
DART Rouge-1	55.1	54.3	39.9	52.7	42.6	39.9	37.6	46.7	43.8	41.8	42.1	41.3	45.1	1.7	29.0
E2ENLG Rouge-1	59.2	58.5	56.0	58.8	53.2	54.7	50.4	59.1	52.3	46.8	47.7	49.3	47.3	4.0	43.7
E2ENLG Rouge-2	34.3	32.9	30.6	33.6	29.6	29.8	27.1	33.2	27.6	23.0	23.6	25.0	23.4	2.3	21.9
E2ENLG Rouge-1	44.8	44.1	41.3	44.4	38.6	39.8	37.7	43.9	37.1	34.5	35.2	37.0	34.7	3.9	32.0
CommonGen Rouge-1	42.4	42.9	27.5	41.1	25.1	26.4	23.3	32.8	24.0	24.7	24.0	25.9	23.3	5.2	17.6
CommonGen Rouge-2	17.2	18.7	8.6	16.9	6.3	8.8	5.6	9.3	6.6	5.5	5.0	6.9	6.4	0.0	5.6
CommonGen Rouge-1	37.8	39.6	24.1	37.0	20.4	21.9	19.1	28.7	20.4	21.1	21.0	21.9	19.8	4.9	14.8
Translation															
WMT'16-tren	3.2	2.7	2.0	3.1	2.6	3.5	3.3	3.7	2.6	3.5	3.2	3.4	3.2	0.0	2.0
WMT'16-deen	18.9	18.4	18.4	18.7	20.3	17.9	18.8	18.8	18.7	11.6	14.0	14.7	16.6	1.1	11.4
WMT'16-ruen	10.8	9.9	10.3	10.4	9.8	9.2	9.3	11.0	10.8	6.2	7.8	8.3	7.3	0.0	4.8
WMT'16-fien	6.5	7.6	7.3	6.5	7.0	7.2	7.1	7.3	7.8	6.2	6.2	6.1	6.5	0.7	4.3
WMT'16-roen	13.9	15.9	12.9	14.0	12.3	12.8	13.3	13.1	12.2	9.8	10.7	10.1	10.3	0.3	8.0
WMT'14-enfr	16.5	16.2	17.1	16.1	16.9	17.7	18.0	17.8	18.0	15.9	17.3	17.1	16.4	3.5	15.2
WMT'16-csen	10.7	7.0	5.2	9.4	7.0	6.1	6.2	8.3	5.8	4.7	6.3	6.3	6.3	0.8	6.1
Paracrawl-enes	24.3	23.2	21.3	24.2	20.3	22.9	22.3	22.8	22.1	18.0	18.8	19.5	21.6	4.5	16.4
COMMONSENSE															
StoryCloze	72.0	78.0	90.0	62.0	42.0	72.0	68.0	84.0	58.0	74.0	70.0	70.0	68.0	62.0	48.0
PIQA	46.0	44.0	38.0	46.0	32.0	34.0	36.0	38.0	34.0	40.0	38.0	38.0	36.0	38.0	0.0
COPA	86.0	62.0	58.0	74.0	68.0	78.0	70.0	80.0	68.0	72.0	70.0	72.0	70.0	56.0	22.0
HellaSwag	46.0	36.0	32.0	40.0	42.0	20.0	18.0	44.0	40.0	32.0	30.0	26.0	26.0	28.0	0.0
sentiment															
SST-2	98.0	96.0	96.0	98.0	96.0	74.0	78.0	96.0	94.0	56.0	68.0	66.0	66.0	74.0	0.0
Yelp	98.0	98.0	98.0	94.0	94.0	96.0	96.0	98.0	98.0	86.0	90.0	86.0	84.0	80.0	0.0
IMDB	96.0	98.0	100.0	96.0	96.0	92.0	82.0	96.0	96.0	76.0	80.0	80.0	84.0	80.0	0.0
sentiment140	68.0	70.0	74.0	70.0	70.0	54.0	58.0	68.0	74.0	62.0	62.0	66.0	62.0	60.0	2.0
READING Comp.															
MultiRC	68.0	64.0	60.0	52.0	38.0	44.0	44.0	48.0	44.0	54.0	52.0	50.0	48.0	40.0	6.0
SQuADv1	68.0	62.0	62.0	66.0	68.0	64.0	64.0	62.0	68.0	68.0	70.0	66.0	66.0	54.0	4.0
SQuADv2	62.0	26.0	8.0	56.0	12.0	30.0	20.0	22.0	16.0	24.0	24.0	26.0	22.0	16.0	0.0
OBQA	82.0	78.0	68.0	68.0	58.0	64.0	60.0	78.0	66.0	62.0	64.0	66.0	60.0	40.0	0.0
BoolQ	84.0	82.0	74.0	60.0	60.0	68.0	70.0	80.0	76.0	74.0	68.0	76.0	70.0	72.0	6.0
Drop	40.0	18.0	16.0	8.0	6.0	14.0	12.0	18.0	14.0	10.0	8.0	8.0	8.0	22.0	0.0
CLOSE-BOOK QA															
NQ	18.0	12.0	10.0	16.0	10.0	16.0	14.0	16.0	10.0	12.0	12.0	12.0	4.0	12.0	0.0
ARC-e	50.0	74.0	80.0	56.0	70.0	54.0	56.0	66.0	82.0	58.0	58.0	60.0	58.0	48.0	0.0
ARC-c	46.0	46.0	48.0	42.0	46.0	34.0	34.0	50.0	46.0	46.0	42.0	42.0	42.0	24.0	0.0
TriviaQa	66.0	56.0	56.0	46.0	46.0	60.0	46.0	48.0	56.0	46.0	42.0	46.0	24.0	42.0	4.0
COREFERENCE															
DPR	54.0	64.0	46.0	50.0	50.0	56.0	60.0	68.0	56.0	64.0	60.0	62.0	62.0	46.0	2.0
WSC	50.0	50.0	56.0	50.0	42.0	38.0	46.0	58.0	42.0	58.0	58.0	52.0	54.0	40.0	0.0
READ. COOMP. W/ COMMONSENSE															
CosmosQa	68.0	74.0	58.0	68.0	34.0	46.0	32.0	50.0	46.0	44.0	46.0	44.0	38.0	14.0	6.0
record	70.0	54.0	30.0	70.0	26.0	24.0	6.0	42.0	34.0	18.0	12.0	14.0	8.0	14.0	0.0
PARAPHRASE															
Paws Wiki	90.0	54.0	44.0	64.0	40.0	44.0	42.0	56.0	46.0	56.0	50.0	48.0	54.0	60.0	2.0
QQP	74.0	88.0	64.0	74.0	68.0	66.0	60.0	80.0	58.0	50.0	40.0	36.0	28.0	54.0	0.0
MRPC	60.0	62.0	60.0	58.0	58.0	60.0	62.0	60.0	58.0	42.0	44.0	40.0	42.0	60.0	2.0
STSBB	38.0	34.0	8.0	36.0	16.0	12.0	12.0	30.0	20.0	20.0	20.0	20.0	14.0	12.0	0.0
NLI															
CB	88.9	82.2	66.7	80.0	62.2	77.8	57.8	86.7	66.7	68.9	64.4	68.9	62.2	55.6	13.3
WNLI	70.0	52.0	42.0	68.0	46.0	44.0	50.0	60.0	54.0	56.0	56.0	42.0	44.0	52.0	0.0
ANLI-r1	50.0	52.0	48.0	50.0	50.0	40.0	42.0	40.0	42.0	40.0	40.0	36.0	38.0	38.0	24.0
ANLI-r3	46.0	50.0	38.0	42.0	38.0	38.0	40.0	44.0	50.0	28.0	32.0	34.0	38.0	40.0	24.0
ANLI-r2	46.0	48.0	46.0	46.0	46.0	32.0	36.0	46.0	46.0	40.0	36.0	38.0	32.0	46.0	20.0
ANLI-m	88.0	76.0	84.0	84.0	88.0	62.0	66.0	80.0	88.0	48.0	54.0	50.0	56.0	76.0	0.0
ANLI-mm	92.0	84.0	90.0	90.0	94.0	64.0	82.0	88.0	90.0	48.0	48.0	50.0	60.0	84.0	2.0
SNLI	96.0	88.0	86.0	84.0	84.0	56.0	58.0	90.0	92.0	54.0	52.0	54.0	54.0	82.0	0.0
QNLI	94.0	66.0	62.0	94.0	26.0	46.0	48.0	74.0	38.0	56.0	56.0	54.0	60.0	70.0	0.0
RTE	52.0	68.0	64.0	62.0	72.0	54.0	58.0	70.0	76.0	64.0	58.0	56.0	64.0	80.0	22.0